Smart Library Management System

A complete beginner's guide to the C code and the data structures inside it

Data Structure Lab Project · Language: C · 11 source files in the src folder

You do not need to know anything about data structures before reading this. Every idea is explained from zero, with pictures, and then matched to the exact lines of code that use it.

Contents

1. What this project is **5**

1.1 The life of one borrowed book . . . 5

1.2 Which data structure is used where . . . 5

1.3 The project files . . . 6

2. How to compile and run **7**

2.1 What you need . . . 7

2.2 Compile and run from a terminal . . . 7

2.3 Using Code::Blocks . . . 7

2.4 First login . . . 7

2.5 Where is the data saved? . . . 8

3. The C you need to know first **9**

3.1 #include and #define . . . 9

3.2 Strings are character arrays . . . 9

3.3 struct - grouping data together . . . 9

3.4 Pointers - the most important idea . . . 10

3.5 malloc and free - making and removing boxes . . . 10

3.6 Global variables . . . 10

3.7 switch - choosing between numbered options . . . 10

3.8 Splitting a program into several files . . . 12

4. Linked list - explained from zero **13**

4.1 The problem with arrays . . . 13

4.2 The idea . . . 13

4.3 Traversal - visiting every node . . . 13

4.4 Searching (linear search) . . . 13

4.5 Inserting at the end . . . 14

4.6 Inserting at the beginning . . . 15

4.7 Deleting a node . . . 16

4.8 Freeing a whole list . . . 16

5. Queue - the borrow request line **17**

5.1 The idea . . . 17

5.2 enqueue - joining the line . . . 17

5.3 dequeue - serving the first in line . . . 18

5.4 Where the queue is used . . . 18

6. Stack - the history **20**

6.1 The idea . . . 20

6.2 push . . . 20

6.3 Reading the stack . . . 21

7. Loans - check-out and check-in **22**

7.1 Check-out . . . 22

7.2 Check-in . . . 23

7.3 Keeping the counts correct . . . 23

8. Walking through every part of the code **24**

8.1 Input helpers . . . 24

8.2 Book helpers . . . 25

8.3 User list . . . 26

8.4 Cart . . . 27

8.5 Request and loan helpers . . . 28

8.6 Librarian actions . . . 30

8.7 Menus and login . . . 32

9. Saving and loading data (file handling) **34**

9.1 The file format . . . 34

9.2 Saving . . . 34

9.3 Loading . . . 35

10. Example session **37**

10.1 Real output from the program . . . 37

11. How fast is each operation? (time complexity) **39**

12. Viva questions and answers **40**

13. Limitations and ideas to improve **42**

14. Glossary **43**

Appendix: the complete source code **44**

A.1 library.h . . . 45

A.2 main.c . . . 48

A.3 menu.c . . . 49

A.4 utils.c . . . 51

A.5 book.c . . . 53

A.6 user.c 56
A.7 cart.c 58
A.8 request.c 60
A.9 loan.c 62
A.10 history.c 64
A.11 file.c 65

1. What this project is

This project is a **library management program** that runs in the terminal (the black text window). There are no buttons or mouse clicks: the program prints a numbered menu, you type a number and press Enter, and the program does that job.

Three kinds of people use it. Each one logs in with a username and password and gets their own menu:

User	What they can do
Student	See all books, search books, put books in a **cart**, send the cart as a **borrow request**, see their requests, **check out** (pick up) a book after it is approved, **check in** (return) a book, see their own history.
Staff	See and search books, check the **stock** of a book, see all pending borrow requests, **approve or reject** them, see the **check-in / check-out history** of any student, see which books are currently with students.
Librarian	**Everything staff can do**, plus: **register** new students and staff, **delete** users, see all users, **add** new books, **delete** books, change the number of copies of a book.

1.1 The life of one borrowed book

The most important idea in the project is the journey a book takes when a student borrows it. Read this once now; every part will be explained in detail later.



- **Cart:** the student collects the books they want (like a shopping cart).
- **Request:** when the student submits the cart, every book becomes a request. Requests wait in a line called a **queue**. The first request sent is the first one staff look at.
- **Approved:** staff approve the request. One copy is now reserved for that student, so the available count goes down by 1.
- **Borrowed:** the student comes and picks it up. This is **check-out**. It is written into the history.
- **Returned:** the student brings it back. This is **check-in**. The available count goes up by 1 and it is written into the history.

If staff **reject** a request, it is removed from the queue and nothing else changes.

1.2 Which data structure is used where

Data structure	Used for	Why this one?
Singly linked list	Books, users, the cart, loans	We do not know in advance how many books or users there will be. A linked list grows and shrinks one item at a time, using exactly as much memory as needed.
Queue (built from a linked list)	Borrow requests	Fairness: first come, first served. The student who asked first is answered first.
Stack (built from a linked list)	Check-in / check-out history	When you look at history you usually want the newest event first. A stack always keeps the newest item on top.

Do not worry if these words mean nothing yet. Chapters 4 to 7 explain each one from zero.

1.3 The project files

The code is split into small files, one file per topic, all inside the **src** folder. When you want to change how books work, you open book.c; when you want to change a menu, you open menu.c.

File	What is inside	Main functions
library.h	The shared "table of contents": #define constants, all structs, the list of global variables and every function's prototype	-
main.c	Where the program starts; the main menu (Login / Exit)	main
menu.c	The three menus, each built with a switch	studentMenu, staffMenu, librarianMenu
utils.c	Small helpers: safe input, date, malloc helper	readLine, readInt, getDate, newNode
book.c	The **book linked list** and book features	addBook, findBook, deleteBookNode, searchBooks, newBook, removeBook
user.c	The **user linked list**, register, delete, login	addUser, findUser, registerUser, login
cart.c	The **cart linked list**	addToCart, removeFromCart, submitCart
request.c	The **request queue** (a linked list)	enqueue, dequeue, processRequests
loan.c	The **loan linked list**, check-out, check-in	addLoan, checkOutBook, checkInBook
history.c	The **history stack** (a linked list)	pushHistory, showHistory
file.c	**File handling**: save everything, load everything	saveAll, loadAll

Three tools are used everywhere in the project, as required for the lab:

- **Linked list** - every list of data (books, users, cart, requests, loans, history) is a linked list made of nodes and pointers. Chapters 4-7.
- **File handling** - all data is saved to and loaded from text files (file.c). Chapter 9.
- **switch** - every place where the user picks from numbered choices uses a switch statement (menus, approve/reject, register role, delete confirm, role names). Section 3.7.

2. How to compile and run

2.1 What you need

- A C compiler. On Windows the easiest is **Code::Blocks** (it comes with the GCC compiler) or **Dev-C++**. On Mac or Linux, **gcc** or **clang** is usually already installed.
- The **src** folder with all 11 files (10 .c files and library.h).

2.2 Compile and run from a terminal

```
cd src                      # go into the folder with the code
gcc *.c -o library          # compile ALL .c files into one program
./library                   # run it   (on Windows type:  library.exe)
```

Compile means: the compiler reads your C code and produces a program the computer can run. `*.c` means "every file ending in .c", so all ten files are compiled together and joined into one program. **-o library** means "name the program *library*". You do not list library.h: it is pulled in by the `#include "library.h"` line at the top of every .c file.

If your terminal does not understand *.c, list the files by hand:

```
gcc main.c menu.c utils.c book.c user.c cart.c request.c loan.c history.c file.c -o library
```

2.3 Using Code::Blocks

- File > New > Project > **Console application** > C. Give it a name and finish.
- Delete the main.c that Code::Blocks made for you (right-click > Remove file from project).
- Copy all 11 files from src into the project folder. Then Project > **Add files...** and select all of them.
- Press **F9** (Build and Run). A console window opens with the main menu.

Common mistake
With several files you must use a **project** in Code::Blocks. If you only open main.c and press F9, you will get errors like *undefined reference to `login'*, because the compiler never saw user.c where login() is written.

Note
Code::Blocks and modern GCC use the C99 standard or newer by default, which this code needs (for example it declares variables inside `for( ... )`). Very old compilers such as Turbo C will not compile it.

2.4 First login

The very first time the program runs there is no data at all, so it creates:

- One librarian account: username **admin**, password **admin123**.
- Five sample books, so you have something to try with.

Log in as admin, register a student and a staff member (librarian menu option 8), log out, and then log in as them to try their menus.

2.5 Where is the data saved?

After every action the program writes everything into five small text files, in the same folder you run it from: **books.txt, users.txt, requests.txt, loans.txt, history.txt**. When you start the program again it reads them back, so nothing is lost. If you delete all five files, the program starts fresh (first run again). Chapter 9 explains this.

3. The C you need to know first

This chapter explains every C feature that the project uses. If you already know C basics, skim it.

3.1 #include and #define

File: library.h (lines 33-37)

```
33  #include <stdio.h>
34  #include <stdlib.h>
35  #include <string.h>
36  #include <ctype.h>
37  #include <time.h>
```

#include brings in ready-made functions. stdio.h gives printf/fgets/fopen, stdlib.h gives malloc/free/exit, string.h gives strcpy/strcmp/strstr, ctype.h gives tolower, time.h gives the current date. These lines are written once in library.h, and every .c file gets them through `#include "library.h"` (section 3.8).

File: library.h (lines 40-42)

```
40  #define ROLE_STUDENT   1
41  #define ROLE_STAFF     2
42  #define ROLE_LIBRARIAN 3
```

#define gives a name to a number. Writing `ROLE_STUDENT` is much clearer than writing `1` everywhere. Before compiling, C replaces every ROLE_STUDENT with 1.

3.2 Strings are character arrays

C has no special "string" type. A name is stored in an array of characters such as `char title[100];` which can hold up to 99 letters plus a hidden end marker `'\0'`. You cannot copy or compare strings with = or ==. Instead:

Function	Meaning	Example from the project
strcpy(a, b)	copy text b into a	strcpy(node->title, title);
strcmp(a, b)	compare; gives 0 when equal	strcmp(temp->username, username) == 0
strstr(a, b)	is b somewhere inside a?	used in containsIgnoreCase()
strchr(a, c)	is the character c in a?	strchr(buf, '|') - forbid the '|' character
strlen(a)	number of characters	strlen(buf) == 0 means empty input

3.3 struct - grouping data together

A book has an id, a title, an author and two counts. Instead of keeping five separate variables, we group them into one **struct**:

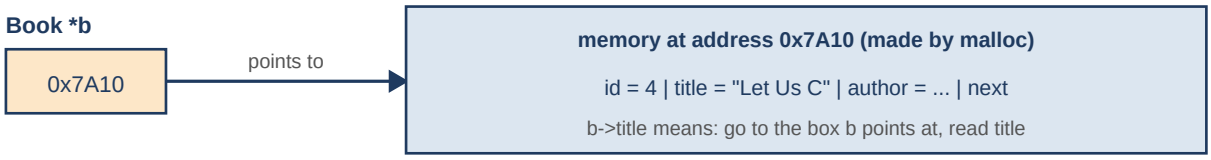
File: library.h (lines 61-68)

```
61  typedef struct Book {
62      int  id;
63      char title[100];
64      char author[60];
65      int  total;        /* how many copies the library owns  */
66      int  available;    /* how many copies are on the shelf  */
67      struct Book *next; /* pointer to the next book          */
68  } Book;
```

typedef lets us write just `Book` instead of `struct Book` later. The interesting line is `struct Book *next;`: every book holds the **address of the next book**. This one line is what turns separate books into a **linked list**.

3.4 Pointers - the most important idea

Every variable lives somewhere in memory, and every place in memory has a number called its **address**. A **pointer** is a variable that stores an address. `Book *b;` means "b holds the address of a Book".



- `b->title`: go to the Book that b points to and use its title. (The arrow `->` is used with pointers, the dot `.` with normal struct variables.)
- `NULL`: a special address meaning "points to nothing". We use it to mark the end of a list and to say "not found".

3.5 malloc and free - making and removing boxes

When a librarian adds a book, the program needs a brand-new Book box in memory while it is running. **malloc** (memory allocate) asks the computer for that memory and returns its address. **free** gives memory back when we delete something. The project puts malloc inside one small helper so that the "out of memory" check is written only once:

File: utils.c (lines 67-76)

```
67  /* Makes a new node with malloc. Stops the program if memory is full. */
68  void *newNode(int size)
69  {
70      void *p = malloc(size);
71      if (p == NULL) {
72          printf("Out of memory!\n");
73          exit(1);
74      }
75      return p;
76  }
```

`sizeof(Book)` tells malloc exactly how many bytes one Book needs. `void *` means "an address of anything", so the same helper can make Books, Users, Requests and the rest.

3.6 Global variables

A global variable is declared outside every function, so every function can use it. Each list's starting pointer is global. It is created in the file that owns that list, for example at the top of book.c:

File: book.c (lines 7-8)

```
7  Book *bookHead   = NULL;   /* first node of the book list */
8  int   nextBookId = 1;      /* id given to the next new book */
```

Each list starts as NULL, meaning "empty". `currentUser` (user.c) remembers who is logged in. `nextBookId` and `nextRequestId` give each new book or request a unique number.

3.7 switch - choosing between numbered options

When the user types a number, the program must do a different job for each number. A long chain of `if ... else if ... else if` works, but a **switch** is cleaner. Here is the main menu:

File: main.c (lines 10-35)

```
10  int main(void)
11  {
12      if (loadAll() == 1) {          /* 1 means: first run, no saved data */
13          createDefaultData();
14      }
15
16      while (1) {
17          printf("\n========================================\n");
18          printf("    SMART LIBRARY MANAGEMENT SYSTEM\n");
19          printf("========================================\n");
20          printf("1. Login\n");
21          printf("0. Exit\n");
22
23          switch (readInt("Choice: ")) {
24              case 1:
25                  login();
26                  break;
27              case 0:
28                  saveAll();
29                  printf("Goodbye!\n");
30                  return 0;
31              default:
32                  printf("Invalid choice.\n");
33          }
34      }
35  }
```

- `switch (readInt("Choice: "))` - read a number and jump to the **case** with that value.
- `case 1:` - runs when the number is 1. `break;` means "this case is finished, leave the switch". Without break, C would fall through and also run the next case.
- `return 0;` inside case 0 ends main(), which ends the whole program.
- `default:` - runs when no case matches (the user typed 7, for example).
- The `while (1)` loop around it shows the menu again and again until the user chooses Exit.

The project uses switch in every place a numbered choice is made:

Where	File	What the switch chooses
main()	main.c	Login or Exit
studentMenu(), staffMenu(), librarianMenu()	menu.c	which feature to run
login()	user.c	which menu to open (by role)
registerUser()	user.c	Student or Staff
processRequests()	request.c	Approve, Reject or Stop
removeBook()	book.c	Yes (delete) or No
roleName(), statusName()	utils.c	which word to print for a number

File: utils.c (lines 78-85)

```
78  const char *roleName(int role)
79  {
80      switch (role) {
81          case ROLE_STUDENT: return "Student";
82          case ROLE_STAFF:   return "Staff";
83          default:           return "Librarian";
84      }
85  }
```

A switch can also **return** straight from a case, as here; then no break is needed.

3.8 Splitting a program into several files

A big program in one file is hard to read. So we split it by topic. But there is a problem: menu.c calls showAllBooks(), which is written in book.c. How does menu.c know that function exists? The answer is a **header file**: library.h.

Prototypes - a list of functions

A **prototype** is the first line of a function followed by a semicolon. It tells the compiler: "this function exists somewhere, it takes these inputs and gives back this type". library.h lists the prototype of every function, grouped by the file it lives in:

File: library.h (lines 140-151)

```
140  /* book.c */
141  Book *findBook(int id);
142  Book *addBook(int id, const char *title, const char *author, int total, int available);
143  int   deleteBookNode(int id);
144  void  printBookHeader(void);
145  void  printBook(Book *b);
146  void  showAllBooks(void);
147  void  searchBooks(void);
148  void  checkStock(void);
149  void  newBook(void);
150  void  removeBook(void);
151  void  updateCopies(void);
```

extern - one variable shared by many files

A global variable must be **created** in exactly one .c file (book.c creates bookHead). Other files only need to know it exists. **extern** says "this variable exists, but it is created in another file":

File: library.h (lines 116-125)

```
116  extern Book     *bookHead;      /* book.c    - first node of book list   */
117  extern int       nextBookId;    /* book.c                                */
118  extern User     *userHead;      /* user.c    - first node of user list   */
119  extern User     *currentUser;   /* user.c    - who is logged in now      */
120  extern CartItem *cartHead;      /* cart.c    - cart of logged-in student */
121  extern Request  *queueFront;    /* request.c - remove from here          */
122  extern Request  *queueRear;     /* request.c - add here                  */
123  extern int       nextRequestId; /* request.c                             */
124  extern Loan     *loanHead;      /* loan.c    - first node of loan list   */
125  extern History  *historyTop;    /* history.c - top of history stack      */
```

#include "library.h" and the include guard

Every .c file starts with `#include "library.h"`. Quotes (not < >) mean "look for this file in my own folder". It is as if library.h were copied into the top of each file, so each file knows every struct, global variable and function.

File: library.h (lines 30-31)

```
30  #ifndef LIBRARY_H      /* "include guard": stops this file from */
31  #define LIBRARY_H      /* being included twice in the same .c   */
```

These two lines (plus `#endif` at the very end of the file) are an **include guard**. If library.h were ever included twice into the same file, the second copy is skipped, so the structs are not defined twice (which would be an error).

How the files become one program
Compiling: each .c file is compiled on its own (it only needs library.h to know about the others). **Linking:** then all the pieces are joined into one program, and every call like showAllBooks() is connected to the real function in book.c. `gcc *.c -o library` does both steps for you.

4. Linked list - explained from zero

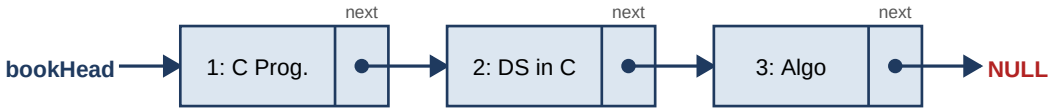
4.1 The problem with arrays

An array like `Book books[100];` has a fixed size. If the library has 101 books, it is full. If it has 3, the memory for 97 is wasted. Deleting book number 2 also means shifting every book after it one place to the left.

4.2 The idea

A **linked list** is a chain of boxes called **nodes**. Each node holds its data and the address of the next node. We only need to remember where the first node is; that pointer is called the **head**. The last node's next is NULL, which means "the chain ends here".

The book list in memory:



Think of a treasure hunt: each clue tells you where the next clue is. You can only start at the first clue and follow them in order. That is exactly how the program moves through a list.

4.3 Traversal - visiting every node

Traversal means walking the chain from head to the end. Every "show" function in the project does this:

File: book.c (lines 70-83)

```
70  /* Traverse (walk through) the whole list and print every book */
71  void showAllBooks(void)
72  {
73      if (bookHead == NULL) {
74          printf("\nNo books in the library.\n");
75          return;
76      }
77      printBookHeader();
78      Book *temp = bookHead;
79      while (temp != NULL) {
80          printBook(temp);
81          temp = temp->next;
82      }
83  }
```

- `temp = bookHead` - start at the first node.
- `while (temp != NULL)` - keep going until we fall off the end.
- `temp = temp->next` - jump to the next node. This line is the heart of every linked list loop.

4.4 Searching (linear search)

To find a book by id we walk the chain and compare each id. If we reach NULL the book is not there. This is called **linear search** because we look at nodes one by one in a line.

File: book.c (lines 10-19)

```
10  /* Linear search: walk the list one by one until the id matches */
11  Book *findBook(int id)
12  {
13      Book *temp = bookHead;
14      while (temp != NULL) {
15          if (temp->id == id) return temp;
16          temp = temp->next;
17      }
18      return NULL;
19  }
```

The same pattern is used by **findUser** (compares usernames with strcmp) and **findLoan**. **searchBooks** is also a linear search, but it prints every book whose title or author contains the typed word, ignoring capital letters:

File: utils.c (lines 48-58)

```
48  /* Returns 1 if 'word' is found inside 'text' (ignoring capital letters) */
49  int containsIgnoreCase(const char *text, const char *word)
50  {
51      char a[100], b[100];
52      int i;
53      for (i = 0; text[i] != '\0' && i < 99; i++) a[i] = tolower((unsigned char)text[i]);
54      a[i] = '\0';
55      for (i = 0; word[i] != '\0' && i < 99; i++) b[i] = tolower((unsigned char)word[i]);
56      b[i] = '\0';
57      return strstr(a, b) != NULL;
58  }
```

It makes small-letter copies of both texts and then asks strstr whether one is inside the other. So searching "data" finds "Data Structures in C".

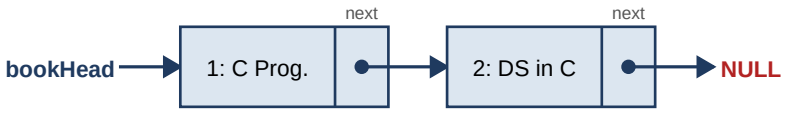
File: book.c (lines 85-103)

```
85  /* Search by title or author (part of the word is enough) */
86  void searchBooks(void)
87  {
88      char key[100];
89      int found = 0;
90      readLine("Enter title or author to search: ", key, sizeof(key));
91
92      Book *temp = bookHead;
93      while (temp != NULL) {
94          if (containsIgnoreCase(temp->title, key) || containsIgnoreCase(temp->author, key)) {
95              if (found == 0) printBookHeader();
96              printBook(temp);
97              found++;
98          }
99          temp = temp->next;
100     }
101     if (found == 0) printf("No book matched \"%s\".\n", key);
102     else printf("%d book(s) found.\n", found);
103 }
```

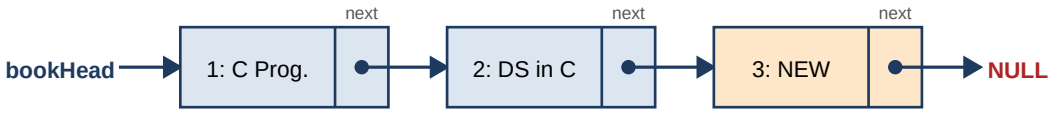
4.5 Inserting at the end

New books and users are added at the end so the list stays in the order they were created.

Before: the last node points to NULL



After: the old last node now points to the new node



File: book.c (lines 21-41)

```
21  /* Insert a new book at the END of the linked list */
22  Book *addBook(int id, const char *title, const char *author, int total, int available)
23  {
24      Book *node = newNode(sizeof(Book));
25      node->id = id;
26      strcpy(node->title, title);
27      strcpy(node->author, author);
28      node->total = total;
29      node->available = available;
30      node->next = NULL;
31
32      if (bookHead == NULL) {          /* list is empty */
33          bookHead = node;
34      } else {                         /* go to the last node */
35          Book *temp = bookHead;
36          while (temp->next != NULL) temp = temp->next;
37          temp->next = node;
38      }
39      if (id >= nextBookId) nextBookId = id + 1;
40      return node;
41  }
```

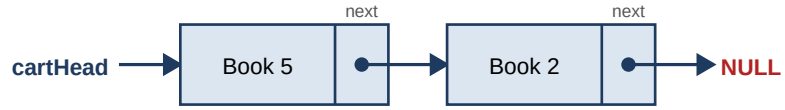
Step by step:

- Make a new node with newNode() and fill in its data. Its next is NULL because it will be last.
- **Empty list?** Then the new node simply becomes the head.
- **Otherwise** walk to the last node (the one whose next is NULL) and point its next at the new node.
- The last `if` keeps nextBookId larger than every id used so far, so ids never repeat - even after loading books from the file.

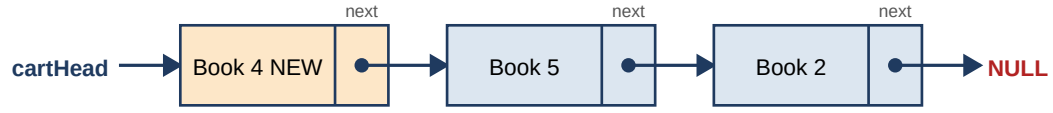
4.6 Inserting at the beginning

The cart does not care about order, so it uses the fastest insert: put the new node in front. Only two lines are needed and no walking at all:

Before



After



File: cart.c (lines 35-37)

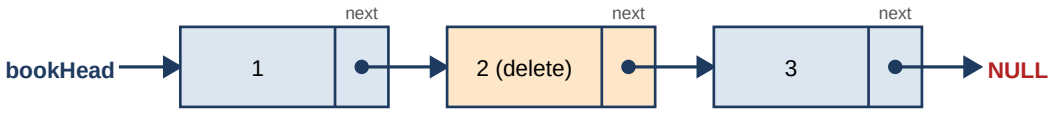
```
35      node->bookId = id;
36      node->next = cartHead;   /* new node points to old first node */
37      cartHead = node;         /* new node becomes the first node   */
```

Order matters: first point the new node at the old first node, *then* move the head. If you swapped these two lines you would lose the rest of the cart.

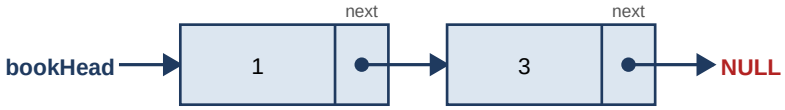
4.7 Deleting a node

To delete, we find the node and remember the one **before** it (prev). Then we make prev skip over it, and free the memory.

Find node 2 - prev is node 1



After: node 1's next points straight to node 3; node 2 is freed



File: book.c (lines 43-57)

```
43  /* Delete the node with this id from the linked list */
44  int deleteBookNode(int id)
45  {
46      Book *temp = bookHead, *prev = NULL;
47      while (temp != NULL && temp->id != id) {
48          prev = temp;
49          temp = temp->next;
50      }
51      if (temp == NULL) return 0;            /* not found */
52
53      if (prev == NULL) bookHead = temp->next;  /* deleting the first node */
54      else prev->next = temp->next;             /* skip over the node      */
55      free(temp);
56      return 1;
57  }
```

- The loop moves prev and temp together, one step at a time.
- If temp becomes NULL, the id was not found: return 0.
- If prev is still NULL, the node to delete is the head, so the head moves to the second node.
- Otherwise `prev->next = temp->next` unlinks the node.
- `free(temp)` returns the memory. Always unlink first, then free.

deleteUserNode and removeFromCart use the exact same steps.

4.8 Freeing a whole list

File: cart.c (lines 74-82)

```
74  /* Free every node of the cart (used after submit and on logout) */
75  void clearCart(void)
76  {
77      while (cartHead != NULL) {
78          CartItem *temp = cartHead;
79          cartHead = cartHead->next;
80          free(temp);
81      }
82  }
```

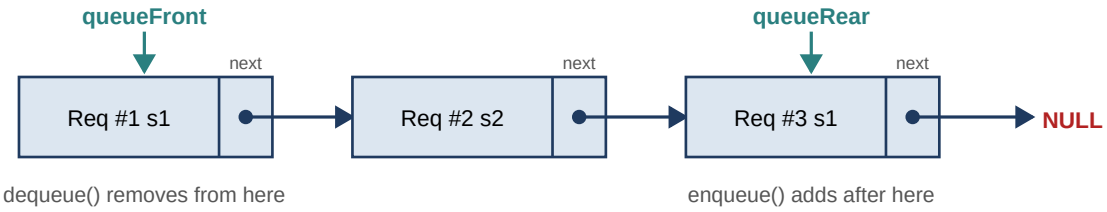
Remove the first node again and again until the list is empty. Notice we save the node in `temp` before moving the head, otherwise we could not free it.

5. Queue - the borrow request line

5.1 The idea

A **queue** is a line of people at a counter. New people join at the **back** (rear), and the person at the **front** is served first. This rule is called **FIFO**: First In, First Out.

Borrow requests are a perfect fit: if two students want the last copy of a book, the one who asked first should get it.



We build the queue from a linked list with **two** pointers: queueFront (who is served next) and queueRear (the last one in line, so we can add behind them without walking the whole list).

File: library.h (lines 86-91)

```
86  typedef struct Request {
87      int  id;
88      char username[30];
89      int  bookId;
90      struct Request *next;
91  } Request;
```

5.2 enqueue - joining the line

File: request.c (lines 11-27)

```
11  void enqueue(int id, const char *username, int bookId)
12  {
13      Request *node = newNode(sizeof(Request));
14      node->id = id;
15      strcpy(node->username, username);
16      node->bookId = bookId;
17      node->next = NULL;
18
19      if (queueRear == NULL) {         /* queue is empty */
20          queueFront = node;
21          queueRear = node;
22      } else {
23          queueRear->next = node;      /* join at the back of the line */
24          queueRear = node;
25      }
26      if (id >= nextRequestId) nextRequestId = id + 1;
27  }
```

- If the queue is empty (rear is NULL), the new request is both the front and the rear.
- Otherwise the old rear points to the new node, and the new node becomes the rear.
- Because we keep a rear pointer, adding is instant - no walking needed.

5.3 dequeue - serving the first in line

File: request.c (lines 29-37)

```
 29  /* Remove and return the request at the front. Caller must free() it. */
 30  Request *dequeue(void)
 31  {
 32      if (queueFront == NULL) return NULL;
 33      Request *node = queueFront;
 34      queueFront = queueFront->next;
 35      if (queueFront == NULL) queueRear = NULL;   /* queue became empty */
 36      return node;
 37  }
```

- If the queue is empty, return NULL (nobody to serve).
- Take the front node, move front to the next node.
- If front became NULL, the queue is now empty, so the rear must become NULL too. Forgetting this line is a classic bug.
- The node is returned (not freed) so the caller can still read who asked for which book. The caller frees it after use.

5.4 Where the queue is used

Student side - submitCart(): every book in the cart is checked, and if it is valid it is enqueued. The checks stop duplicates: you cannot request the same book twice, or request a book you already have.

File: cart.c (lines 84-107)

```
 84  /* Every book in the cart becomes a request in the queue */
 85  void submitCart(void)
 86  {
 87      if (cartHead == NULL) {
 88          printf("Your cart is empty. Add books first.\n");
 89          return;
 90      }
 91      CartItem *temp = cartHead;
 92      while (temp != NULL) {
 93          Book *b = findBook(temp->bookId);
 94          if (b == NULL) {
 95              printf("Book %d no longer exists - skipped.\n", temp->bookId);
 96          } else if (hasPendingRequest(currentUser->username, temp->bookId)) {
 97              printf("You already requested \"%s\" - skipped.\n", b->title);
 98          } else if (hasActiveLoan(currentUser->username, temp->bookId)) {
 99              printf("You already have \"%s\" - skipped.\n", b->title);
100          } else {
101              enqueue(nextRequestId, currentUser->username, temp->bookId);
102              printf("Request sent for \"%s\".\n", b->title);
103          }
104          temp = temp->next;
105      }
106      clearCart();
107  }
```

Staff side - processRequests(): staff always look at the front of the queue. Approve or reject dequeues it; Stop leaves it there for later.

File: request.c (lines 70-108)

```
/* Staff: take requests from the FRONT of the queue one by one */
void processRequests(void)
{
    while (queueFront != NULL) {
        Request *r = queueFront;
        Book *b = findBook(r->bookId);

        printf("\nNext request  #%d\n", r->id);
        printf("  Student : %s\n", r->username);
        printf("  Book    : %d - %s\n", r->bookId, b ? b->title : "(deleted book)");
        printf("  Copies available: %d\n", b ? b->available : 0);
        printf("1. Approve   2. Reject   0. Stop\n");

        switch (readInt("Choice: ")) {
            case 1:                                 /* approve */
                if (b == NULL || b->available == 0) {
                    printf("Cannot approve: no copy available. Reject it or try later.\n");
                    break;
                }
                r = dequeue();
                b->available--;                     /* reserve one copy */
                addLoan(r->id, r->username, r->bookId, STATUS_APPROVED);
                printf("Approved. Student can now check out the book.\n");
                free(r);
                break;
            case 2:                                 /* reject */
                r = dequeue();
                printf("Request rejected.\n");
                free(r);
                break;
            case 0:                                 /* stop */
                return;
            default:
                printf("Invalid choice.\n");
        }
        saveAll();
    }
    printf("\nNo more pending requests.\n");
}
```

- **Approve** only works if a copy is available. Then available goes down by 1 (the copy is reserved), and a **loan** with status APPROVED is created using the same id as the request.
- **Reject** just removes the request.
- If no copy is free, the request stays at the front; staff can reject it or stop and try later.
- saveAll() is called after each decision so nothing is lost.

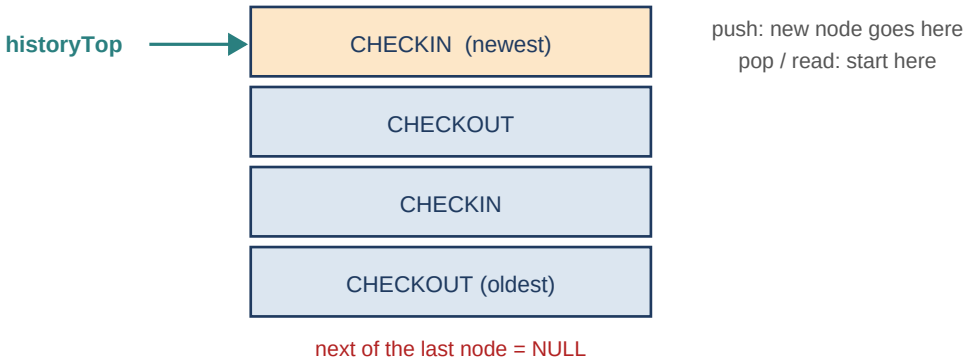
showRequests(name) walks the queue from front to rear (it never removes anything). Passing NULL shows everyone's requests (staff); passing a username shows only that student's (student menu).

6. Stack - the history

6.1 The idea

A **stack** is a pile of plates. You put a new plate on **top** (push), and you take the top plate first (pop). This rule is called **LIFO**: Last In, First Out.

For history this is exactly what we want: the newest check-in or check-out is on top, so when staff open the history they see the most recent events first.



File: library.h (lines 103-109)

```
103  typedef struct History {
104      char username[30];
105      int  bookId;
106      char action[10];   /* "CHECKOUT" or "CHECKIN" */
107      char date[20];
108      struct History *next;
109  } History;
```

6.2 push

File: history.c (lines 8-17)

```
  8  void pushHistory(const char *username, int bookId, const char *action, const char *date)
  9  {
 10      History *node = newNode(sizeof(History));
 11      strcpy(node->username, username);
 12      node->bookId = bookId;
 13      strcpy(node->action, action);
 14      strcpy(node->date, date);
 15      node->next = historyTop;   /* new node sits on top of old top */
 16      historyTop = node;
 17  }
```

It is the same as "insert at the beginning" of a linked list: the new node points at the old top, then becomes the new top. push is called in two places: checkOutBook() writes "CHECKOUT" and checkInBook() writes "CHECKIN", each with the current date from getDate().

6.3 Reading the stack

File: history.c (lines 19-39)

```
19  /* Show history from top of stack (newest) to bottom (oldest).
20   * username == NULL means "show everyone's". */
21  void showHistory(const char *username)
22  {
23      int count = 0;
24      History *temp = historyTop;
25      while (temp != NULL) {
26          if (username == NULL || strcmp(temp->username, username) == 0) {
27              if (count == 0) {
28                  printf("\n%-17s %-15s %-9s %-8s %-25s\n", "Date", "Student", "Action", "BookID", "Title");
29                  printf("-------------------------------------------------------------------------------------\n");
30              }
31              Book *b = findBook(temp->bookId);
32              printf("%-17s %-15s %-9s %-8d %-25.25s\n", temp->date, temp->username,
33                     temp->action, temp->bookId, b ? b->title : "(deleted book)");
34              count++;
35          }
36          temp = temp->next;
37      }
38      if (count == 0) printf("\nNo history found.\n");
39  }
```

Reading from top to bottom gives newest-first order automatically. Like the queue, NULL means "everyone", and a username shows only that student. We never pop (remove) history, because a library should keep its records.

7. Loans - check-out and check-in

A **loan** remembers that a particular student has (or had) a particular book. Loans are kept in a normal linked list. Each loan has a status:

Status	Value	Meaning
STATUS_APPROVED	1	Staff approved. One copy is reserved. Student has not picked it up yet.
STATUS_BORROWED	2	Student checked it out and has the book.
STATUS_RETURNED	3	Student checked it in. Kept as a record.

File: library.h (lines 94-100)

```
 94  typedef struct Loan {
 95      int  id;
 96      char username[30];
 97      int  bookId;
 98      int  status;
 99      struct Loan *next;
100  } Loan;
```

7.1 Check-out

File: loan.c (lines 83-102)

```
 83  /* Student picks up an approved book */
 84  void checkOutBook(void)
 85  {
 86      showLoans(currentUser->username, 1);
 87      int id = readInt("Enter Loan ID to check out (pick up): ");
 88      Loan *loan = findLoan(id);
 89      if (loan == NULL || strcmp(loan->username, currentUser->username) != 0) {
 90          printf("No such loan for you.\n");
 91          return;
 92      }
 93      if (loan->status != STATUS_APPROVED) {
 94          printf("This loan is not waiting for pick up.\n");
 95          return;
 96      }
 97      char date[20];
 98      getDate(date, sizeof(date));
 99      loan->status = STATUS_BORROWED;
100      pushHistory(currentUser->username, loan->bookId, "CHECKOUT", date);
101      printf("Book checked out. Enjoy reading!\n");
102  }
```

- Show the student's active loans so they can see the Loan ID.
- Find the loan. It must exist, belong to this student, and be APPROVED.
- Change the status to BORROWED and push a CHECKOUT record on the history stack.
- The available count does not change here - it already went down when staff approved.

7.2 Check-in

File: loan.c (lines 104-125)

```
104  /* Student returns a borrowed book */
105  void checkInBook(void)
106  {
107      showLoans(currentUser->username, 1);
108      int id = readInt("Enter Loan ID to check in (return): ");
109      Loan *loan = findLoan(id);
110      if (loan == NULL || strcmp(loan->username, currentUser->username) != 0) {
111          printf("No such loan for you.\n");
112          return;
113      }
114      if (loan->status != STATUS_BORROWED) {
115          printf("This book is not checked out.\n");
116          return;
117      }
118      char date[20];
119      getDate(date, sizeof(date));
120      loan->status = STATUS_RETURNED;
121      Book *b = findBook(loan->bookId);
122      if (b != NULL) b->available++;          /* copy goes back on the shelf */
123      pushHistory(currentUser->username, loan->bookId, "CHECKIN", date);
124      printf("Book checked in. Thank you!\n");
125  }
```

Same checks, but the loan must be BORROWED. The status becomes RETURNED, the book's available count goes up by 1 (the copy is back on the shelf), and a CHECKIN record is pushed.

7.3 Keeping the counts correct

Each book has **total** (copies the library owns) and **available** (copies on the shelf). The rule is always: **total - available = copies that are approved or borrowed**. Every function that touches the counts respects this rule:

Action	available	Why
Staff approves	- 1	copy reserved for the student
Student checks out	no change	already reserved
Student checks in	+ 1	copy back on the shelf
Librarian changes total	total - out	can never go below copies that are out
Librarian deletes book	-	only allowed if available == total (no copy is out)

8. Walking through every part of the code

The file is organised in sections, each marked with a big comment banner. This chapter goes through every function that was not already explained, in the order it appears in the file.

8.1 Input helpers

Reading input safely is surprisingly important. If a program uses `scanf("%d")` and the user types a letter, the program can loop forever. This project reads every input as a whole line with **fgets**, then checks it.

File: utils.c (lines 6-32)

```
 6  /* Read one line of text. Never empty, and never contains '|'
 7   * (because '|' is used to separate fields inside our files). */
 8  void readLine(const char *prompt, char *buf, int size)
 9  {
10      while (1) {
11          printf("%s", prompt);
12          if (fgets(buf, size, stdin) == NULL) {   /* input ended (Ctrl+D) */
13              printf("\nInput closed. Saving and exiting.\n");
14              saveAll();
15              exit(0);
16          }
17          char *nl = strchr(buf, '\n');
18          if (nl != NULL) {
19              *nl = '\0';                 /* remove the Enter key   */
20          } else {
21              int c;                       /* line too long: throw  */
22              while ((c = getchar()) != '\n' && c != EOF) { } /* the rest away */
23          }
24          if (strlen(buf) == 0) {
25              printf("  Input cannot be empty. Try again.\n");
26          } else if (strchr(buf, '|') != NULL) {
27              printf("  The character '|' is not allowed. Try again.\n");
28          } else {
29              return;
30          }
31      }
32  }
```

- **fgets** reads up to size-1 characters, including the Enter key ('\n').
- If fgets returns NULL, input has ended (for example Ctrl+D). We save and exit instead of looping forever.
- If a '\n' is found we replace it with '\0' to remove it. If not, the user typed more than the buffer holds, so we throw away the rest of the line with getchar().
- Empty input and the '|' character are refused (the '|' is used inside our save files to separate fields, so allowing it would break them).

File: utils.c (lines 34-46)

```
34  /* Read a whole number (int). Keeps asking until the user types one. */
35  int readInt(const char *prompt)
36  {
37      char buf[50];
38      int value;
39      while (1) {
40          readLine(prompt, buf, sizeof(buf));
41          if (sscanf(buf, "%d", &value) == 1) {
42              return value;
43          }
44          printf("  Please enter a number.\n");
45      }
46  }
```

readInt uses readLine and then **sscanf** to turn the text into a number. sscanf returns how many values it read; if it is not 1, the user did not type a number, so we ask again.

File: utils.c (lines 60-65)

```
60  /* Puts today's date and time into buf, like "2026-11-10 14:30" */
61  void getDate(char *buf, int size)
62  {
63      time_t now = time(NULL);
64      strftime(buf, size, "%Y-%m-%d %H:%M", localtime(&now));
65  }
```

time(NULL) gives the current time as a number; localtime breaks it into year, month, day...; strftime formats it as text like 2026-11-10 14:30.

File: utils.c (lines 78-85)

```
78  const char *roleName(int role)
79  {
80      switch (role) {
81          case ROLE_STUDENT: return "Student";
82          case ROLE_STAFF:   return "Staff";
83          default:           return "Librarian";
84      }
85  }
```

File: utils.c (lines 87-94)

```
87  const char *statusName(int status)
88  {
89      switch (status) {
90          case STATUS_APPROVED: return "Approved (pick up)";
91          case STATUS_BORROWED: return "Borrowed";
92          default:              return "Returned";
93      }
94  }
```

Two tiny helpers that turn a number into a word for printing.

8.2 Book helpers

File: book.c (lines 59-63)

```
59  void printBookHeader(void)
60  {
61      printf("\n%-5s %-32s %-22s %-6s %-6s\n", "ID", "Title", "Author", "Total", "Avail");
62      printf("--------------------------------------------------------------------------------\n");
63  }
```

File: book.c (lines 65-68)

```
65  void printBook(Book *b)
66  {
67      printf("%-5d %-32.32s %-22.22s %-6d %-6d\n", b->id, b->title, b->author, b->total, b->available);
68  }
```

`%-32.32s` means: print text left-aligned in 32 spaces, and cut it if it is longer than 32. This keeps the table columns straight.

File: book.c (lines 105-117)

```
105  /* Staff: check how many copies of one book are in stock */
106  void checkStock(void)
107  {
108      int id = readInt("Enter book ID: ");
109      Book *b = findBook(id);
110      if (b == NULL) {
111          printf("Book not found.\n");
112          return;
113      }
114      printBookHeader();
115      printBook(b);
116      printf("Borrowed / reserved copies: %d\n", b->total - b->available);
117  }
```

Staff type a book ID and see its total, available, and how many copies are out.

8.3 User list

File: library.h (lines 71-77)

```
71  typedef struct User {
72      char username[30];
73      char password[30];
74      char name[60];
75      int  role;
76      struct User *next;
77  } User;
```

File: user.c (lines 10-18)

```
10  User *findUser(const char *username)
11  {
12      User *temp = userHead;
13      while (temp != NULL) {
14          if (strcmp(temp->username, username) == 0) return temp;
15          temp = temp->next;
16      }
17      return NULL;
18  }
```

File: user.c (lines 20-37)

```
20  /* Insert a user at the END of the user list */
21  void addUser(const char *username, const char *password, const char *name, int role)
22  {
23      User *node = newNode(sizeof(User));
24      strcpy(node->username, username);
25      strcpy(node->password, password);
26      strcpy(node->name, name);
27      node->role = role;
28      node->next = NULL;
29
30      if (userHead == NULL) {
31          userHead = node;
32      } else {
33          User *temp = userHead;
34          while (temp->next != NULL) temp = temp->next;
35          temp->next = node;
36      }
37  }
```

Exactly like findBook and addBook, but comparing usernames with strcmp.

File: user.c (lines 39-51)

```
39  int deleteUserNode(const char *username)
40  {
41      User *temp = userHead, *prev = NULL;
42      while (temp != NULL && strcmp(temp->username, username) != 0) {
43          prev = temp;
44          temp = temp->next;
45      }
46      if (temp == NULL) return 0;
47      if (prev == NULL) userHead = temp->next;
48      else prev->next = temp->next;
49      free(temp);
50      return 1;
51  }
```

File: user.c (lines 53-62)

```
53  void showAllUsers(void)
54  {
55      printf("\n%-15s %-25s %-10s\n", "Username", "Name", "Role");
56      printf("-------------------------------------------------\n");
57      User *temp = userHead;
58      while (temp != NULL) {
59          printf("%-15s %-25.25s %-10s\n", temp->username, temp->name, roleName(temp->role));
60          temp = temp->next;
61      }
62  }
```

8.4 Cart

File: library.h (lines 80-83)

```
80  typedef struct CartItem {
81      int bookId;
82      struct CartItem *next;
83  } CartItem;
```

The cart only stores book ids, not whole books - the book's details are looked up with findBook when needed. The cart belongs to the session: it is emptied when the student logs out and is not saved to a file.

File: cart.c (lines 8-16)

```
 8  int cartContains(int bookId)
 9  {
10      CartItem *temp = cartHead;
11      while (temp != NULL) {
12          if (temp->bookId == bookId) return 1;
13          temp = temp->next;
14      }
15      return 0;
16  }
```

File: cart.c (lines 18-39)

```
18  /* Insert at the BEGINNING of the cart list (fastest insert) */
19  void addToCart(void)
20  {
21      int id = readInt("Enter book ID to add to cart: ");
22      Book *b = findBook(id);
23      if (b == NULL) {
24          printf("Book not found.\n");
25          return;
26      }
27      if (cartContains(id)) {
28          printf("This book is already in your cart.\n");
29          return;
30      }
31      if (b->available == 0) {
32          printf("Note: no copy is free right now, but you can still request it.\n");
33      }
34      CartItem *node = newNode(sizeof(CartItem));
35      node->bookId = id;
36      node->next = cartHead;   /* new node points to old first node */
37      cartHead = node;         /* new node becomes the first node   */
38      printf("\"%s\" added to cart.\n", b->title);
39  }
```

File: cart.c (lines 41-57)

```
41  void removeFromCart(void)
42  {
43      int id = readInt("Enter book ID to remove from cart: ");
44      CartItem *temp = cartHead, *prev = NULL;
45      while (temp != NULL && temp->bookId != id) {
46          prev = temp;
47          temp = temp->next;
48      }
49      if (temp == NULL) {
50          printf("That book is not in your cart.\n");
51          return;
52      }
53      if (prev == NULL) cartHead = temp->next;
54      else prev->next = temp->next;
55      free(temp);
56      printf("Removed from cart.\n");
57  }
```

File: cart.c (lines 59-72)

```
59  void showCart(void)
60  {
61      if (cartHead == NULL) {
62          printf("\nYour cart is empty.\n");
63          return;
64      }
65      printBookHeader();
66      CartItem *temp = cartHead;
67      while (temp != NULL) {
68          Book *b = findBook(temp->bookId);
69          if (b != NULL) printBook(b);
70          temp = temp->next;
71      }
72  }
```

8.5 Request and loan helpers

File: request.c (lines 39-58)

```
39  /* Show pending requests. username == NULL means "show everyone's". */
40  void showRequests(const char *username)
41  {
42      int count = 0;
43      Request *temp = queueFront;
44      while (temp != NULL) {
45          if (username == NULL || strcmp(temp->username, username) == 0) {
46              if (count == 0) {
47                  printf("\n%-6s %-15s %-8s %-32s\n", "ReqID", "Student", "BookID", "Title");
48                  printf("-----------------------------------------------------------------\n");
49              }
50              Book *b = findBook(temp->bookId);
51              printf("%-6d %-15s %-8d %-32.32s\n", temp->id, temp->username,
52                     temp->bookId, b ? b->title : "(deleted book)");
53              count++;
54          }
55          temp = temp->next;
56      }
57      if (count == 0) printf("\nNo pending requests.\n");
58  }
```

File: request.c (lines 60-68)

```
60  int hasPendingRequest(const char *username, int bookId)
61  {
62      Request *temp = queueFront;
63      while (temp != NULL) {
64          if (strcmp(temp->username, username) == 0 && temp->bookId == bookId) return 1;
65          temp = temp->next;
66      }
67      return 0;
68  }
```

File: loan.c (lines 8-25)

```
 8  void addLoan(int id, const char *username, int bookId, int status)
 9  {
10      Loan *node = newNode(sizeof(Loan));
11      node->id = id;
12      strcpy(node->username, username);
13      node->bookId = bookId;
14      node->status = status;
15      node->next = NULL;
16
17      if (loanHead == NULL) {
18          loanHead = node;
19      } else {
20          Loan *temp = loanHead;
21          while (temp->next != NULL) temp = temp->next;
22          temp->next = node;
23      }
24      if (id >= nextRequestId) nextRequestId = id + 1;
25  }
```

File: loan.c (lines 27-35)

```
Loan *findLoan(int id)
{
    Loan *temp = loanHead;
    while (temp != NULL) {
        if (temp->id == id) return temp;
        temp = temp->next;
    }
    return NULL;
}
```

File: loan.c (lines 37-47)

```
/* Returns 1 if this user still has this book (approved or borrowed) */
int hasActiveLoan(const char *username, int bookId)
{
    Loan *temp = loanHead;
    while (temp != NULL) {
        if (strcmp(temp->username, username) == 0 && temp->bookId == bookId &&
            temp->status != STATUS_RETURNED) return 1;
        temp = temp->next;
    }
    return 0;
}
```

File: loan.c (lines 60-81)

```
/* Show loans. username == NULL means everyone. onlyActive == 1 hides returned ones. */
void showLoans(const char *username, int onlyActive)
{
    int count = 0;
    Loan *temp = loanHead;
    while (temp != NULL) {
        int userOk   = (username == NULL || strcmp(temp->username, username) == 0);
        int statusOk = (!onlyActive || temp->status != STATUS_RETURNED);
        if (userOk && statusOk) {
            if (count == 0) {
                printf("\n%-7s %-15s %-8s %-28s %-18s\n", "LoanID", "Student", "BookID", "Title", "Status");
                printf("-------------------------------------------------------------------------------\n");
            }
            Book *b = findBook(temp->bookId);
            printf("%-7d %-15s %-8d %-28.28s %-18s\n", temp->id, temp->username, temp->bookId,
                   b ? b->title : "(deleted book)", statusName(temp->status));
            count++;
        }
        temp = temp->next;
    }
    if (count == 0) printf("\nNo loans found.\n");
}
```

showLoans(username, onlyActive) has two filters. The student menu option "My books" shows all their loans including returned ones; the check-in / check-out screens and staff's "books currently with students" pass onlyActive = 1 to hide returned loans.

File: history.c (lines 41-51)

```
/* Staff: see the history of one student, or of everyone */
void viewStudentHistory(void)
{
    char username[30];
    readLine("Enter student username (or ALL): ", username, sizeof(username));
    if (strcmp(username, "ALL") == 0 || strcmp(username, "all") == 0) {
        showHistory(NULL);
    } else {
        showHistory(username);
    }
}
```

8.6 Librarian actions

File: user.c (lines 64-90)

```
 64  /* Librarian: register a new student or staff member */
 65  void registerUser(void)
 66  {
 67      char username[30], password[30], name[60];
 68      int role;
 69
 70      printf("\nRegister as:  1. Student   2. Staff\n");
 71      switch (readInt("Choice: ")) {
 72          case 1:  role = ROLE_STUDENT; break;
 73          case 2:  role = ROLE_STAFF;   break;
 74          default: printf("Invalid role.\n"); return;
 75      }
 76
 77      readLine("Username (no spaces): ", username, sizeof(username));
 78      if (strchr(username, ' ') != NULL) {
 79          printf("Username cannot contain spaces.\n");
 80          return;
 81      }
 82      if (findUser(username) != NULL) {
 83          printf("This username is already taken.\n");
 84          return;
 85      }
 86      readLine("Password: ", password, sizeof(password));
 87      readLine("Full name: ", name, sizeof(name));
 88      addUser(username, password, name, role);
 89      printf("%s \"%s\" registered successfully.\n", roleName(role), username);
 90  }
```

- The librarian chooses Student (1) or Staff (2). Only these two roles can be created.
- The username must have no spaces and must not already exist (checked with findUser).
- Then addUser puts the new person at the end of the user list.

File: user.c (lines 92-117)

```
 92  /* Librarian: delete a student or staff member */
 93  void removeUser(void)
 94  {
 95      char username[30];
 96      showAllUsers();
 97      readLine("Enter username to delete: ", username, sizeof(username));
 98      User *u = findUser(username);
 99      if (u == NULL) {
100          printf("User not found.\n");
101          return;
102      }
103      if (u == currentUser) {
104          printf("You cannot delete yourself.\n");
105          return;
106      }
107      if (u->role == ROLE_LIBRARIAN) {
108          printf("A librarian account cannot be deleted.\n");
109          return;
110      }
111      if (userHasActiveLoans(username)) {
112          printf("This user still has books. They must return them first.\n");
113          return;
114      }
115      deleteUserNode(username);
116      printf("User deleted.\n");
117  }
```

Safety checks before deleting: the user must exist, cannot be the person logged in, cannot be a librarian, and must not have any book that is approved or borrowed. Only then is the node removed.

File: loan.c (lines 49-58)

```
/* Returns 1 if this user still has ANY book (approved or borrowed) */
int userHasActiveLoans(const char *username)
{
    Loan *temp = loanHead;
    while (temp != NULL) {
        if (strcmp(temp->username, username) == 0 && temp->status != STATUS_RETURNED) return 1;
        temp = temp->next;
    }
    return 0;
}
```

This helper (in loan.c) walks the loan list and returns 1 as soon as it finds a loan of this user that is still APPROVED or BORROWED.

File: book.c (lines 119-132)

```
/* Librarian: add a brand new book */
void newBook(void)
{
    char title[100], author[60];
    readLine("Book title: ", title, sizeof(title));
    readLine("Author: ", author, sizeof(author));
    int copies = readInt("Number of copies: ");
    if (copies <= 0) {
        printf("Copies must be at least 1.\n");
        return;
    }
    Book *b = addBook(nextBookId, title, author, copies, copies);
    printf("Book added with ID %d.\n", b->id);
}
```

File: book.c (lines 134-156)

```
/* Librarian: delete a book (only when no copy is out) */
void removeBook(void)
{
    int id = readInt("Enter book ID to delete: ");
    Book *b = findBook(id);
    if (b == NULL) {
        printf("Book not found.\n");
        return;
    }
    if (b->available != b->total) {
        printf("Some copies are borrowed or reserved. Cannot delete now.\n");
        return;
    }
    printf("Delete \"%s\"? 1. Yes  0. No\n", b->title);
    switch (readInt("Choice: ")) {
        case 1:
            deleteBookNode(id);
            printf("Book deleted.\n");
            break;
        default:
            printf("Nothing deleted.\n");
    }
}
```

A book can only be deleted when every copy is on the shelf (available == total), and the librarian must confirm with 1.

File: book.c (lines 158-177)

```
158  /* Librarian: change how many copies the library owns */
159  void updateCopies(void)
160  {
161      int id = readInt("Enter book ID: ");
162      Book *b = findBook(id);
163      if (b == NULL) {
164          printf("Book not found.\n");
165          return;
166      }
167      int out = b->total - b->available;    /* copies currently out */
168      printf("Total now: %d  (out with students: %d)\n", b->total, out);
169      int newTotal = readInt("New total copies: ");
170      if (newTotal < out) {
171          printf("Total cannot be less than copies that are out (%d).\n", out);
172          return;
173      }
174      b->total = newTotal;
175      b->available = newTotal - out;
176      printf("Updated. Available now: %d\n", b->available);
177  }
```

`out` = copies with students. The new total cannot be smaller than that, and available is recalculated as newTotal - out.

8.7 Menus and login

File: menu.c (lines 7-41)

```
 7  void studentMenu(void)
 8  {
 9      while (1) {
10          printf("\n===== STUDENT MENU (%s) =====\n", currentUser->name);
11          printf(" 1. Show all books\n");
12          printf(" 2. Search book\n");
13          printf(" 3. Add book to cart\n");
14          printf(" 4. Remove book from cart\n");
15          printf(" 5. View cart\n");
16          printf(" 6. Send borrow request (submit cart)\n");
17          printf(" 7. My pending requests\n");
18          printf(" 8. My books (loans)\n");
19          printf(" 9. Check out (pick up approved book)\n");
20          printf("10. Check in (return book)\n");
21          printf("11. My history\n");
22          printf(" 0. Logout\n");
23
24          switch (readInt("Choice: ")) {
25              case 1:  showAllBooks(); break;
26              case 2:  searchBooks(); break;
27              case 3:  addToCart(); break;
28              case 4:  removeFromCart(); break;
29              case 5:  showCart(); break;
30              case 6:  submitCart(); break;
31              case 7:  showRequests(currentUser->username); break;
32              case 8:  showLoans(currentUser->username, 0); break;
33              case 9:  checkOutBook(); break;
34              case 10: checkInBook(); break;
35              case 11: showHistory(currentUser->username); break;
36              case 0:  clearCart(); return;
37              default: printf("Invalid choice.\n");
38          }
39          saveAll();
40      }
41  }
```

Every menu has the same shape: an endless **while (1)** loop prints the options, reads a choice, and a **switch** calls the right function. Choice 0 uses **return** to leave the loop and go back to login. After every action saveAll() writes the data to the files.

staffMenu and librarianMenu are built the same way. The librarian menu repeats the seven staff options (calling the very same functions - no duplicated logic) and adds six more.

File: user.c (lines 119-142)

```
119  /* Ask for username + password, then open the right menu */
120  void login(void)
121  {
122      char username[30], password[30];
123      readLine("Username: ", username, sizeof(username));
124      readLine("Password: ", password, sizeof(password));
125
126      User *u = findUser(username);
127      if (u == NULL || strcmp(u->password, password) != 0) {
128          printf("Wrong username or password.\n");
129          return;
130      }
131      currentUser = u;
132      printf("\nWelcome, %s! You are logged in as %s.\n", u->name, roleName(u->role));
133
134      switch (u->role) {
135          case ROLE_STUDENT: studentMenu();   break;
136          case ROLE_STAFF:   staffMenu();     break;
137          default:           librarianMenu(); break;
138      }
139
140      currentUser = NULL;
141      printf("Logged out.\n");
142  }
```

Find the user by username and compare the password. If they match, remember them in currentUser and open the menu that matches their role.

main (main.c, shown in section 3.7) is where every C program starts. It loads the saved data (or creates the default data on the first run) and then shows the main menu until the user chooses Exit.

9. Saving and loading data (file handling)

Variables live in memory, and memory is wiped when the program closes. To remember books and users between runs we write them to text files.

9.1 The file format

Each node becomes one line. Fields are separated by the **|** character (that is why readLine refuses '|'). For example books.txt looks like this:

```
1|C Programming Language|Kernighan & Ritchie|3|3
2|Data Structures in C|Reema Thareja|2|1
4|Let Us C|Yashavant Kanetkar|4|4
```

File	One line means	Fields
books.txt	one book	id | title | author | total | available
users.txt	one user	username | password | name | role
requests.txt	one pending request (front first)	id | username | bookId
loans.txt	one loan	id | username | bookId | status
history.txt	one history record (newest first)	username | bookId | action | date

9.2 Saving

File: file.c (lines 7-46)

```
 7  void saveAll(void)
 8  {
 9      FILE *f;
10
11      f = fopen(BOOK_FILE, "w");
12      if (f != NULL) {
13          for (Book *b = bookHead; b != NULL; b = b->next)
14              fprintf(f, "%d|%s|%s|%d|%d\n", b->id, b->title, b->author, b->total, b->available);
15          fclose(f);
16      }
17
18      f = fopen(USER_FILE, "w");
19      if (f != NULL) {
20          for (User *u = userHead; u != NULL; u = u->next)
21              fprintf(f, "%s|%s|%s|%d\n", u->username, u->password, u->name, u->role);
22          fclose(f);
23      }
24
25      f = fopen(REQUEST_FILE, "w");
26      if (f != NULL) {
27          for (Request *r = queueFront; r != NULL; r = r->next)
28              fprintf(f, "%d|%s|%d\n", r->id, r->username, r->bookId);
29          fclose(f);
30      }
31
32      f = fopen(LOAN_FILE, "w");
33      if (f != NULL) {
34          for (Loan *l = loanHead; l != NULL; l = l->next)
35              fprintf(f, "%d|%s|%d|%d\n", l->id, l->username, l->bookId, l->status);
36          fclose(f);
37      }
38
39      /* History is saved from top (newest) to bottom (oldest) */
40      f = fopen(HISTORY_FILE, "w");
41      if (f != NULL) {
42          for (History *h = historyTop; h != NULL; h = h->next)
43              fprintf(f, "%s|%d|%s|%s\n", h->username, h->bookId, h->action, h->date);
44          fclose(f);
45      }
46  }
```

- **fopen(name, "w")** opens a file for writing (it erases the old content). If it returns NULL, the file could not be opened, so we skip it instead of crashing.
- Each `for` loop is just a list traversal; **fprintf** works like printf but writes into the file.
- **fclose** finishes writing and closes the file.

9.3 Loading

File: file.c (lines 48-117)

```
 48  /* Returns 1 if this is the very first run (no users file yet) */
 49  int loadAll(void)
 50  {
 51      FILE *f;
 52      char line[300];
 53
 54      f = fopen(USER_FILE, "r");
 55      if (f == NULL) return 1;              /* first run */
 56      while (fgets(line, sizeof(line), f)) {
 57          char username[30], password[30], name[60];
 58          int role;
 59          if (sscanf(line, "%29[^|]|%29[^|]|%59[^|]|%d", username, password, name, &role) == 4)
 60              addUser(username, password, name, role);
 61      }
 62      fclose(f);
 63
 64      f = fopen(BOOK_FILE, "r");
 65      if (f != NULL) {
 66          while (fgets(line, sizeof(line), f)) {
 67              int id, total, available;
 68              char title[100], author[60];
 69              if (sscanf(line, "%d|%99[^|]|%59[^|]|%d|%d", &id, title, author, &total, &available) == 5)
 70                  addBook(id, title, author, total, available);
 71          }
 72          fclose(f);
 73      }
 74
 75      f = fopen(REQUEST_FILE, "r");
 76      if (f != NULL) {
 77          while (fgets(line, sizeof(line), f)) {
```

```
 78              int id, bookId;
 79              char username[30];
 80              if (sscanf(line, "%d|%29[^|]|%d", &id, username, &bookId) == 3)
 81                  enqueue(id, username, bookId);
 82          }
 83          fclose(f);
 84      }
 85
 86      f = fopen(LOAN_FILE, "r");
 87      if (f != NULL) {
 88          while (fgets(line, sizeof(line), f)) {
 89              int id, bookId, status;
 90              char username[30];
 91              if (sscanf(line, "%d|%29[^|]|%d|%d", &id, username, &bookId, &status) == 4)
 92                  addLoan(id, username, bookId, status);
 93          }
 94          fclose(f);
 95      }
 96
 97      /* The file is newest-first. If we pushed each line, the oldest would end
 98       * up on top (wrong order). So we add each line at the BOTTOM instead. */
 99      f = fopen(HISTORY_FILE, "r");
100      if (f != NULL) {
101          History *bottom = NULL;
102          while (fgets(line, sizeof(line), f)) {
103              History *node = newNode(sizeof(History));
104              if (sscanf(line, "%29[^|]|%d|%9[^|]|%19[^\n]", node->username, &node->bookId,
105                         node->action, node->date) != 4) {
106                  free(node);
107                  continue;
```

```
108             }
109             node->next = NULL;
110             if (bottom == NULL) historyTop = node;
111             else bottom->next = node;
112             bottom = node;
113         }
114         fclose(f);
115     }
116     return 0;
117 }
```

- **fopen(name, "r")** opens for reading. If users.txt does not exist this is the first run, so the function returns 1 and main() calls createDefaultData().
- **fgets** reads one line at a time.
- **sscanf** splits the line. `%29[^|]` means "read up to 29 characters that are not '|'". This lets titles and names contain spaces. sscanf returns how many fields it read; broken lines are skipped.
- Books, users and loans are loaded with the normal add functions, so they keep their order. Requests are enqueued, so the front of the queue is still the oldest request.

A small trick in loadAll
History was saved newest-first. If we pushed each line while loading, the first line (the newest) would end up at the bottom - the order would flip every time you restart. So for history the loader adds each line at the **bottom** (keeping a `bottom` pointer), which rebuilds the stack in exactly the same order.

File: file.c (lines 119-130)

```
119 /* First run: create the main librarian and a few sample books */
120 void createDefaultData(void)
121 {
122     addUser("admin", "admin123", "Head Librarian", ROLE_LIBRARIAN);
123     addBook(nextBookId, "C Programming Language", "Kernighan & Ritchie", 3, 3);
124     addBook(nextBookId, "Data Structures in C", "Reema Thareja", 2, 2);
125     addBook(nextBookId, "Introduction to Algorithms", "Cormen", 2, 2);
126     addBook(nextBookId, "Let Us C", "Yashavant Kanetkar", 4, 4);
127     addBook(nextBookId, "Clean Code", "Robert C. Martin", 1, 1);
128     saveAll();
129     printf("First run: default librarian created (admin / admin123).\n");
130 }
```

10. Example session

Here is a complete example of the borrowing journey, in order:

#	Who	Menu option	What happens inside
1	admin (Librarian)	8 Register - Student s1	addUser() adds s1 at the end of the user list
2	admin	8 Register - Staff t1	addUser() adds t1
3	s1 (Student)	2 Search "data"	linear search over the book list
4	s1	3 Add book 2 to cart	insert at the beginning of the cart list
5	s1	6 Submit cart	enqueue(): request #1 joins the rear of the queue; cart cleared
6	t1 (Staff)	5 Approve	dequeue() front; book 2 available 2 -> 1; loan #1 APPROVED
7	s1	9 Check out loan 1	status BORROWED; push CHECKOUT on the history stack
8	s1	10 Check in loan 1	status RETURNED; available 1 -> 2; push CHECKIN
9	t1	6 History of s1	read the stack top to bottom: CHECKIN, then CHECKOUT

10.1 Real output from the program

This is the real text the program printed while testing exactly these steps (menus are shortened):

```
>>> admin logs in and registers student s1 and staff t1
Username: Password:
Welcome, Head Librarian! You are logged in as Librarian.
Register as:  1. Student   2. Staff
Username (no spaces): Password: Full name: Student "s1" registered successfully.
Register as:  1. Student   2. Staff
Username (no spaces): Password: Full name: Staff "t1" registered successfully.

>>> s1 searches 'data', adds book 2 to cart, submits it
Username: Password:
Welcome, Rahim Uddin! You are logged in as Student.
Enter title or author to search:
ID    Title                           Author                Total  Avail
------------------------------------------------------------------------
2     Data Structures in C            Reema Thareja         2      2
1 book(s) found.
Enter book ID to add to cart: "Data Structures in C" added to cart.
Request sent for "Data Structures in C".

>>> t1 approves the request at the front of the queue
Username: Password:
Welcome, Karim Ahmed! You are logged in as Staff.
Next request  #1
  Student : s1
  Book    : 2 - Data Structures in C
  Copies available: 2
Approved. Student can now check out the book.
No more pending requests.

>>> s1 checks out loan 1, then checks it in
```

```
Username: Password:
Welcome, Rahim Uddin! You are logged in as Student.
LoanID  Student         BookID   Title                         Status
-------------------------------------------------------------------------------
1       s1              2        Data Structures in C          Approved (pick up)
Enter Loan ID to check out (pick up): Book checked out. Enjoy reading!
LoanID  Student         BookID   Title                         Status
-------------------------------------------------------------------------------
1       s1              2        Data Structures in C          Borrowed
Enter Loan ID to check in (return): Book checked in. Thank you!

>>> t1 views the history of s1 (newest first)
Username: Password:
Welcome, Karim Ahmed! You are logged in as Staff.
Enter student username (or ALL):
Date              Student         Action    BookID   Title
-------------------------------------------------------------------------------
2026-11-10 09:43  s1              CHECKIN   2        Data Structures in C
2026-11-10 09:43  s1              CHECKOUT  2        Data Structures in C
```

11. How fast is each operation? (time complexity)

Programmers describe speed with **Big-O**. **O(1)** means "always takes the same short time, no matter how much data". **O(n)** means "takes longer as the list grows, because we may have to visit all n nodes".

Operation	Function	Time	Why
Find book / user / loan	findBook, findUser, findLoan	O(n)	linear search walks the list
Add book / user / loan	addBook, addUser, addLoan	O(n)	walks to the last node
Delete book / user	deleteBookNode, deleteUserNode	O(n)	must find it first
Add to cart	addToCart	O(n)	insert is O(1), but the duplicate check and findBook walk
Enqueue request	enqueue	O(1)	we keep a rear pointer
Dequeue request	dequeue	O(1)	front is always known
Push history	pushHistory	O(1)	add on top
Show anything	show...	O(n)	must print every node

For a college library with hundreds or a few thousand books, O(n) is instant on a modern computer. Adding a tail pointer to the book list would make addBook O(1); a binary search tree or hash table would make searching faster - good ideas for a future version.

12. Viva questions and answers

Q. What is a data structure?

A way of organising data in memory so it can be used efficiently. This project uses linked lists, a queue and a stack.

Q. Why a linked list instead of an array?

The number of books and users is unknown and changes. A linked list grows one node at a time with malloc and shrinks with free, and deleting does not require shifting elements.

Q. What is a node?

One struct in the list: the data plus a pointer called next to the following node.

Q. What does head = NULL mean?

The list is empty.

Q. How do you know you reached the end of a list?

The current pointer becomes NULL (the last node's next is NULL).

Q. What is the difference between a queue and a stack?

A queue is FIFO (first in, first out): requests are served in the order they arrive. A stack is LIFO (last in, first out): the newest history record is on top.

Q. Why does the queue have two pointers?

front to remove quickly and rear to add quickly. Both are O(1).

Q. What happens in dequeue when the last element is removed?

front becomes NULL, so rear must also be set to NULL, otherwise rear would point to freed memory.

Q. Why is the book list insert at the end but the cart insert at the beginning?

Books should be shown in the order they were added (by id). Cart order does not matter, and inserting at the beginning is O(1).

Q. Why do we need prev when deleting?

In a singly linked list you cannot go backwards, so we keep the previous node to change its next pointer and skip the deleted node.

Q. What is the difference between malloc and free?

malloc reserves memory for a new node and returns its address; free gives that memory back when the node is deleted.

Q. What is the difference between . and ->?

Use . with a struct variable and -> with a pointer to a struct. p->x is the same as (*p).x.

Q. When is the stock (available) reduced?

When staff approve the request, so the copy is reserved even before the student picks it up.

Q. Why fgets instead of scanf?

fgets reads a full line safely (with a size limit), handles names with spaces, and wrong input cannot leave junk that makes the menu loop forever.

Q. How is data saved?

saveAll() traverses each list and writes one line per node to a text file with fprintf; loadAll() reads lines with fgets and splits them with sscanf.

Q. Why does loadAll not use pushHistory?

The file is newest-first; pushing each line would reverse the order. Adding at the bottom rebuilds the same stack.

Q. What is the time complexity of searching a book?

O(n), linear search.

13. Limitations and ideas to improve

- **Passwords are stored as plain text** in users.txt. Fine for a lab, but a real system would store a hash of the password.
- Searching is linear (O(n)). A **binary search tree** on book id or title would be faster for huge libraries.
- No due dates or fines. You could store a due date in each loan and calculate a fine at check-in.
- The cart is not saved when a student logs out.
- Only one program can use the files at a time (no network / multiple computers).
- Ideas for extra marks: sort books by title (bubble sort on the linked list), a due-date reminder, a per-student borrow limit, or a doubly linked list so lists can be shown in reverse.

14. Glossary

Word	Meaning
Address	The location number of a value in memory.
Pointer	A variable that stores an address. Book *b;
NULL	A pointer value meaning "points to nothing".
Node	One element of a linked list: data + next pointer.
Head	Pointer to the first node of a list.
Traverse	Visit every node from head to the end.
Linear search	Check items one by one until you find the one you want.
Queue / FIFO	First In, First Out. Add at rear, remove at front.
Enqueue / Dequeue	Add to a queue / remove from a queue.
Stack / LIFO	Last In, First Out. Add and remove at the top.
Push / Pop	Add to the top of a stack / remove from the top.
malloc / free	Ask for memory for a new node / give memory back.
struct	A group of variables under one name.
typedef	Gives a type a shorter name.
Traversal pointer (temp)	A helper pointer that moves along a list so the head is not lost.
Big-O	A way to describe how running time grows as data grows.
File handling	Saving to and reading from files with fopen, fprintf, fgets, fclose.

Appendix: the complete source code

Every line of every file in src, with line numbers. The line numbers used throughout this guide refer to these listings (each code box says which file and lines it comes from).

A.1 library.h

File: library.h (lines 1-201)

```
/*
 * ============================================================
 *        SMART LIBRARY MANAGEMENT SYSTEM (SLMS)
 *        Data Structure Lab Project - written in C
 * ============================================================
 *
 * library.h  -  the shared "table of contents" of the project.
 *
 * Every .c file starts with  #include "library.h"  so that it
 * knows about all structures, global variables and functions,
 * even the ones written in other files.
 *
 * Project files:
 *   library.h  - structures, constants, function list (this file)
 *   main.c     - program starts here (main menu)
 *   menu.c     - student, staff and librarian menus
 *   utils.c    - input helpers and small tools
 *   book.c     - book linked list
 *   user.c     - user linked list, register, login
 *   cart.c     - cart linked list
 *   request.c  - borrow request queue (linked list)
 *   loan.c     - loan linked list, check-out, check-in
 *   history.c  - history stack (linked list)
 *   file.c     - save to files and load from files
 *
 * Compile : gcc *.c -o library
 * Run     : ./library        (Windows: library.exe)
 */

#ifndef LIBRARY_H      /* "include guard": stops this file from */
#define LIBRARY_H      /* being included twice in the same .c   */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include <time.h>

/* ---------- Roles of users ---------- */
#define ROLE_STUDENT   1
#define ROLE_STAFF     2
#define ROLE_LIBRARIAN 3

/* ---------- Status of a loan ---------- */
#define STATUS_APPROVED 1   /* staff approved, student has not picked it up yet */
#define STATUS_BORROWED 2   /* student checked the book out                     */
#define STATUS_RETURNED 3   /* student checked the book back in                 */

/* ---------- File names (data is saved here) ---------- */
#define BOOK_FILE    "books.txt"
#define USER_FILE    "users.txt"
#define REQUEST_FILE "requests.txt"
#define LOAN_FILE    "loans.txt"
#define HISTORY_FILE "history.txt"

/* ============================================================
 *                  STRUCTURES (NODES)
 * ============================================================ */

/* One book = one node of the BOOK LINKED LIST */
typedef struct Book {
    int  id;
    char title[100];
    char author[60];
    int  total;        /* how many copies the library owns  */
    int  available;    /* how many copies are on the shelf  */
    struct Book *next; /* pointer to the next book          */
} Book;

/* One user = one node of the USER LINKED LIST */
typedef struct User {
    char username[30];
    char password[30];
    char name[60];
```

```
    int  role;
    struct User *next;
} User;

/* One cart item = one node of the CART LINKED LIST */
typedef struct CartItem {
    int bookId;
    struct CartItem *next;
} CartItem;

/* One borrow request = one node of the REQUEST QUEUE */
typedef struct Request {
    int  id;
    char username[30];
    int  bookId;
    struct Request *next;
} Request;

/* One loan = one node of the LOAN LINKED LIST */
typedef struct Loan {
    int  id;
    char username[30];
    int  bookId;
    int  status;
    struct Loan *next;
} Loan;

/* One history record = one node of the HISTORY STACK */
typedef struct History {
    char username[30];
    int  bookId;
    char action[10];   /* "CHECKOUT" or "CHECKIN" */
    char date[20];
    struct History *next;
} History;

/* ============================================================
 *  GLOBAL VARIABLES
 *  "extern" means: this variable exists, but it is created
 *  (defined) in another .c file. Written next to each one.
 * ============================================================ */
extern Book     *bookHead;      /* book.c    - first node of book list   */
extern int       nextBookId;    /* book.c                                */
extern User     *userHead;      /* user.c    - first node of user list   */
extern User     *currentUser;   /* user.c    - who is logged in now      */
extern CartItem *cartHead;      /* cart.c    - cart of logged-in student */
extern Request  *queueFront;    /* request.c - remove from here          */
extern Request  *queueRear;     /* request.c - add here                  */
extern int       nextRequestId; /* request.c                             */
extern Loan     *loanHead;      /* loan.c    - first node of loan list   */
extern History  *historyTop;    /* history.c - top of history stack      */

/* ============================================================
 *  FUNCTION LIST (prototypes) - grouped by the file they live in
 * ============================================================ */

/* utils.c */
void  readLine(const char *prompt, char *buf, int size);
int   readInt(const char *prompt);
int   containsIgnoreCase(const char *text, const char *word);
void  getDate(char *buf, int size);
void *newNode(int size);
const char *roleName(int role);
const char *statusName(int status);

/* book.c */
Book *findBook(int id);
Book *addBook(int id, const char *title, const char *author, int total, int available);
int   deleteBookNode(int id);
void  printBookHeader(void);
void  printBook(Book *b);
void  showAllBooks(void);
void  searchBooks(void);
void  checkStock(void);
```

```
void  newBook(void);
void  removeBook(void);
void  updateCopies(void);

/* user.c */
User *findUser(const char *username);
void  addUser(const char *username, const char *password, const char *name, int role);
int   deleteUserNode(const char *username);
void  showAllUsers(void);
void  registerUser(void);
void  removeUser(void);
void  login(void);

/* cart.c */
int   cartContains(int bookId);
void  addToCart(void);
void  removeFromCart(void);
void  showCart(void);
void  clearCart(void);
void  submitCart(void);

/* request.c */
void     enqueue(int id, const char *username, int bookId);
Request *dequeue(void);
void     showRequests(const char *username);
int      hasPendingRequest(const char *username, int bookId);
void     processRequests(void);

/* loan.c */
void  addLoan(int id, const char *username, int bookId, int status);
Loan *findLoan(int id);
int   hasActiveLoan(const char *username, int bookId);
int   userHasActiveLoans(const char *username);
void  showLoans(const char *username, int onlyActive);
void  checkOutBook(void);
void  checkInBook(void);

/* history.c */
void  pushHistory(const char *username, int bookId, const char *action, const char *date);
void  showHistory(const char *username);
void  viewStudentHistory(void);

/* file.c */
void  saveAll(void);
int   loadAll(void);
void  createDefaultData(void);

/* menu.c */
void  studentMenu(void);
void  staffMenu(void);
void  librarianMenu(void);

#endif
```

A.2 main.c

File: main.c (lines 1-35)

```
 1  /*
 2   * main.c - the program starts here
 3   *
 4   * Compile : gcc *.c -o library
 5   * Run     : ./library        (Windows: library.exe)
 6   * First login: username = admin   password = admin123
 7   */
 8  #include "library.h"
 9
10  int main(void)
11  {
12      if (loadAll() == 1) {          /* 1 means: first run, no saved data */
13          createDefaultData();
14      }
15
16      while (1) {
17          printf("\n========================================\n");
18          printf("    SMART LIBRARY MANAGEMENT SYSTEM\n");
19          printf("========================================\n");
20          printf("1. Login\n");
21          printf("0. Exit\n");
22
23          switch (readInt("Choice: ")) {
24              case 1:
25                  login();
26                  break;
27              case 0:
28                  saveAll();
29                  printf("Goodbye!\n");
30                  return 0;
31              default:
32                  printf("Invalid choice.\n");
33          }
34      }
35  }
```

A.3 menu.c

File: menu.c (lines 1-111)

```
/*
 * menu.c - the three menus. Each one: print options, read a number,
 *          and a switch calls the right function.
 */
#include "library.h"

void studentMenu(void)
{
    while (1) {
        printf("\n===== STUDENT MENU (%s) =====\n", currentUser->name);
        printf(" 1. Show all books\n");
        printf(" 2. Search book\n");
        printf(" 3. Add book to cart\n");
        printf(" 4. Remove book from cart\n");
        printf(" 5. View cart\n");
        printf(" 6. Send borrow request (submit cart)\n");
        printf(" 7. My pending requests\n");
        printf(" 8. My books (loans)\n");
        printf(" 9. Check out (pick up approved book)\n");
        printf("10. Check in (return book)\n");
        printf("11. My history\n");
        printf(" 0. Logout\n");

        switch (readInt("Choice: ")) {
            case 1:  showAllBooks(); break;
            case 2:  searchBooks(); break;
            case 3:  addToCart(); break;
            case 4:  removeFromCart(); break;
            case 5:  showCart(); break;
            case 6:  submitCart(); break;
            case 7:  showRequests(currentUser->username); break;
            case 8:  showLoans(currentUser->username, 0); break;
            case 9:  checkOutBook(); break;
            case 10: checkInBook(); break;
            case 11: showHistory(currentUser->username); break;
            case 0:  clearCart(); return;
            default: printf("Invalid choice.\n");
        }
        saveAll();
    }
}

void staffMenu(void)
{
    while (1) {
        printf("\n===== STAFF MENU (%s) =====\n", currentUser->name);
        printf(" 1. Show all books\n");
        printf(" 2. Search book\n");
        printf(" 3. Check stock of a book\n");
        printf(" 4. View pending borrow requests\n");
        printf(" 5. Approve / reject requests\n");
        printf(" 6. View check-in / check-out history\n");
        printf(" 7. View books currently with students\n");
        printf(" 0. Logout\n");

        switch (readInt("Choice: ")) {
            case 1: showAllBooks(); break;
            case 2: searchBooks(); break;
            case 3: checkStock(); break;
            case 4: showRequests(NULL); break;
            case 5: processRequests(); break;
            case 6: viewStudentHistory(); break;
            case 7: showLoans(NULL, 1); break;
            case 0: return;
            default: printf("Invalid choice.\n");
        }
        saveAll();
    }
}

void librarianMenu(void)
{
    while (1) {
        printf("\n===== LIBRARIAN MENU (%s) =====\n", currentUser->name);
```

```
 75          printf("--- Staff features ---\n");
 76          printf(" 1. Show all books\n");
 77          printf(" 2. Search book\n");
 78          printf(" 3. Check stock of a book\n");
 79          printf(" 4. View pending borrow requests\n");
 80          printf(" 5. Approve / reject requests\n");
 81          printf(" 6. View check-in / check-out history\n");
 82          printf(" 7. View books currently with students\n");
 83          printf("--- Librarian features ---\n");
 84          printf(" 8. Register new student / staff\n");
 85          printf(" 9. Delete a user\n");
 86          printf("10. Show all users\n");
 87          printf("11. Add new book\n");
 88          printf("12. Delete book\n");
 89          printf("13. Update number of copies\n");
 90          printf(" 0. Logout\n");
 91
 92          switch (readInt("Choice: ")) {
 93              case 1:  showAllBooks(); break;
 94              case 2:  searchBooks(); break;
 95              case 3:  checkStock(); break;
 96              case 4:  showRequests(NULL); break;
 97              case 5:  processRequests(); break;
 98              case 6:  viewStudentHistory(); break;
 99              case 7:  showLoans(NULL, 1); break;
100              case 8:  registerUser(); break;
101              case 9:  removeUser(); break;
102              case 10: showAllUsers(); break;
103              case 11: newBook(); break;
104              case 12: removeBook(); break;
105              case 13: updateCopies(); break;
106              case 0:  return;
107              default: printf("Invalid choice.\n");
108          }
109          saveAll();
110      }
111  }
```

A.4 utils.c

File: utils.c (lines 1-94)

```
/*
 * utils.c - small helper functions used by every other file
 */
#include "library.h"

/* Read one line of text. Never empty, and never contains '|'
 * (because '|' is used to separate fields inside our files). */
void readLine(const char *prompt, char *buf, int size)
{
    while (1) {
        printf("%s", prompt);
        if (fgets(buf, size, stdin) == NULL) {   /* input ended (Ctrl+D) */
            printf("\nInput closed. Saving and exiting.\n");
            saveAll();
            exit(0);
        }
        char *nl = strchr(buf, '\n');
        if (nl != NULL) {
            *nl = '\0';                 /* remove the Enter key   */
        } else {
            int c;                       /* line too long: throw  */
            while ((c = getchar()) != '\n' && c != EOF) { } /* the rest away */
        }
        if (strlen(buf) == 0) {
            printf("  Input cannot be empty. Try again.\n");
        } else if (strchr(buf, '|') != NULL) {
            printf("  The character '|' is not allowed. Try again.\n");
        } else {
            return;
        }
    }
}

/* Read a whole number (int). Keeps asking until the user types one. */
int readInt(const char *prompt)
{
    char buf[50];
    int value;
    while (1) {
        readLine(prompt, buf, sizeof(buf));
        if (sscanf(buf, "%d", &value) == 1) {
            return value;
        }
        printf("  Please enter a number.\n");
    }
}

/* Returns 1 if 'word' is found inside 'text' (ignoring capital letters) */
int containsIgnoreCase(const char *text, const char *word)
{
    char a[100], b[100];
    int i;
    for (i = 0; text[i] != '\0' && i < 99; i++) a[i] = tolower((unsigned char)text[i]);
    a[i] = '\0';
    for (i = 0; word[i] != '\0' && i < 99; i++) b[i] = tolower((unsigned char)word[i]);
    b[i] = '\0';
    return strstr(a, b) != NULL;
}

/* Puts today's date and time into buf, like "2026-11-10 14:30" */
void getDate(char *buf, int size)
{
    time_t now = time(NULL);
    strftime(buf, size, "%Y-%m-%d %H:%M", localtime(&now));
}

/* Makes a new node with malloc. Stops the program if memory is full. */
void *newNode(int size)
{
    void *p = malloc(size);
    if (p == NULL) {
        printf("Out of memory!\n");
        exit(1);
    }
```

```
75      return p;
76  }
77
78  const char *roleName(int role)
79  {
80      switch (role) {
81          case ROLE_STUDENT: return "Student";
82          case ROLE_STAFF:   return "Staff";
83          default:           return "Librarian";
84      }
85  }
86
87  const char *statusName(int status)
88  {
89      switch (status) {
90          case STATUS_APPROVED: return "Approved (pick up)";
91          case STATUS_BORROWED: return "Borrowed";
92          default:              return "Returned";
93      }
94  }
```

A.5 book.c

File: book.c (lines 1-177)

```
/*
 * book.c - the BOOK LINKED LIST
 *          (add, find, delete, show, search, stock, update)
 */
#include "library.h"

Book *bookHead   = NULL;   /* first node of the book list */
int   nextBookId = 1;      /* id given to the next new book */

/* Linear search: walk the list one by one until the id matches */
Book *findBook(int id)
{
    Book *temp = bookHead;
    while (temp != NULL) {
        if (temp->id == id) return temp;
        temp = temp->next;
    }
    return NULL;
}

/* Insert a new book at the END of the linked list */
Book *addBook(int id, const char *title, const char *author, int total, int available)
{
    Book *node = newNode(sizeof(Book));
    node->id = id;
    strcpy(node->title, title);
    strcpy(node->author, author);
    node->total = total;
    node->available = available;
    node->next = NULL;

    if (bookHead == NULL) {          /* list is empty */
        bookHead = node;
    } else {                         /* go to the last node */
        Book *temp = bookHead;
        while (temp->next != NULL) temp = temp->next;
        temp->next = node;
    }
    if (id >= nextBookId) nextBookId = id + 1;
    return node;
}

/* Delete the node with this id from the linked list */
int deleteBookNode(int id)
{
    Book *temp = bookHead, *prev = NULL;
    while (temp != NULL && temp->id != id) {
        prev = temp;
        temp = temp->next;
    }
    if (temp == NULL) return 0;            /* not found */

    if (prev == NULL) bookHead = temp->next;  /* deleting the first node */
    else prev->next = temp->next;             /* skip over the node      */
    free(temp);
    return 1;
}

void printBookHeader(void)
{
    printf("\n%-5s %-32s %-22s %-6s %-6s\n", "ID", "Title", "Author", "Total", "Avail");
    printf("-------------------------------------------------------------------------------\n");
}

void printBook(Book *b)
{
    printf("%-5d %-32.32s %-22.22s %-6d %-6d\n", b->id, b->title, b->author, b->total, b->available);
}

/* Traverse (walk through) the whole list and print every book */
void showAllBooks(void)
{
    if (bookHead == NULL) {
        printf("\nNo books in the library.\n");
```

```
        return;
    }
    printBookHeader();
    Book *temp = bookHead;
    while (temp != NULL) {
        printBook(temp);
        temp = temp->next;
    }
}

/* Search by title or author (part of the word is enough) */
void searchBooks(void)
{
    char key[100];
    int found = 0;
    readLine("Enter title or author to search: ", key, sizeof(key));

    Book *temp = bookHead;
    while (temp != NULL) {
        if (containsIgnoreCase(temp->title, key) || containsIgnoreCase(temp->author, key)) {
            if (found == 0) printBookHeader();
            printBook(temp);
            found++;
        }
        temp = temp->next;
    }
    if (found == 0) printf("No book matched \"%s\".\n", key);
    else printf("%d book(s) found.\n", found);
}

/* Staff: check how many copies of one book are in stock */
void checkStock(void)
{
    int id = readInt("Enter book ID: ");
    Book *b = findBook(id);
    if (b == NULL) {
        printf("Book not found.\n");
        return;
    }
    printBookHeader();
    printBook(b);
    printf("Borrowed / reserved copies: %d\n", b->total - b->available);
}

/* Librarian: add a brand new book */
void newBook(void)
{
    char title[100], author[60];
    readLine("Book title: ", title, sizeof(title));
    readLine("Author: ", author, sizeof(author));
    int copies = readInt("Number of copies: ");
    if (copies <= 0) {
        printf("Copies must be at least 1.\n");
        return;
    }
    Book *b = addBook(nextBookId, title, author, copies, copies);
    printf("Book added with ID %d.\n", b->id);
}

/* Librarian: delete a book (only when no copy is out) */
void removeBook(void)
{
    int id = readInt("Enter book ID to delete: ");
    Book *b = findBook(id);
    if (b == NULL) {
        printf("Book not found.\n");
        return;
    }
    if (b->available != b->total) {
        printf("Some copies are borrowed or reserved. Cannot delete now.\n");
        return;
    }
    printf("Delete \"%s\"? 1. Yes  0. No\n", b->title);
    switch (readInt("Choice: ")) {
```

```
        case 1:
            deleteBookNode(id);
            printf("Book deleted.\n");
            break;
        default:
            printf("Nothing deleted.\n");
    }
}

/* Librarian: change how many copies the library owns */
void updateCopies(void)
{
    int id = readInt("Enter book ID: ");
    Book *b = findBook(id);
    if (b == NULL) {
        printf("Book not found.\n");
        return;
    }
    int out = b->total - b->available;    /* copies currently out */
    printf("Total now: %d  (out with students: %d)\n", b->total, out);
    int newTotal = readInt("New total copies: ");
    if (newTotal < out) {
        printf("Total cannot be less than copies that are out (%d).\n", out);
        return;
    }
    b->total = newTotal;
    b->available = newTotal - out;
    printf("Updated. Available now: %d\n", b->available);
}
```

A.6 user.c

File: user.c (lines 1-142)

```
/*
 * user.c - the USER LINKED LIST (add, find, delete, show)
 *          plus register, remove and login
 */
#include "library.h"

User *userHead    = NULL;   /* first node of the user list */
User *currentUser = NULL;   /* who is logged in now        */

User *findUser(const char *username)
{
    User *temp = userHead;
    while (temp != NULL) {
        if (strcmp(temp->username, username) == 0) return temp;
        temp = temp->next;
    }
    return NULL;
}

/* Insert a user at the END of the user list */
void addUser(const char *username, const char *password, const char *name, int role)
{
    User *node = newNode(sizeof(User));
    strcpy(node->username, username);
    strcpy(node->password, password);
    strcpy(node->name, name);
    node->role = role;
    node->next = NULL;

    if (userHead == NULL) {
        userHead = node;
    } else {
        User *temp = userHead;
        while (temp->next != NULL) temp = temp->next;
        temp->next = node;
    }
}

int deleteUserNode(const char *username)
{
    User *temp = userHead, *prev = NULL;
    while (temp != NULL && strcmp(temp->username, username) != 0) {
        prev = temp;
        temp = temp->next;
    }
    if (temp == NULL) return 0;
    if (prev == NULL) userHead = temp->next;
    else prev->next = temp->next;
    free(temp);
    return 1;
}

void showAllUsers(void)
{
    printf("\n%-15s %-25s %-10s\n", "Username", "Name", "Role");
    printf("-------------------------------------------------------\n");
    User *temp = userHead;
    while (temp != NULL) {
        printf("%-15s %-25.25s %-10s\n", temp->username, temp->name, roleName(temp->role));
        temp = temp->next;
    }
}

/* Librarian: register a new student or staff member */
void registerUser(void)
{
    char username[30], password[30], name[60];
    int role;

    printf("\nRegister as:  1. Student   2. Staff\n");
    switch (readInt("Choice: ")) {
        case 1:  role = ROLE_STUDENT; break;
        case 2:  role = ROLE_STAFF;   break;
        default: printf("Invalid role.\n"); return;
```

```
    }

    readLine("Username (no spaces): ", username, sizeof(username));
    if (strchr(username, ' ') != NULL) {
        printf("Username cannot contain spaces.\n");
        return;
    }
    if (findUser(username) != NULL) {
        printf("This username is already taken.\n");
        return;
    }
    readLine("Password: ", password, sizeof(password));
    readLine("Full name: ", name, sizeof(name));
    addUser(username, password, name, role);
    printf("%s \"%s\" registered successfully.\n", roleName(role), username);
}

/* Librarian: delete a student or staff member */
void removeUser(void)
{
    char username[30];
    showAllUsers();
    readLine("Enter username to delete: ", username, sizeof(username));
    User *u = findUser(username);
    if (u == NULL) {
        printf("User not found.\n");
        return;
    }
    if (u == currentUser) {
        printf("You cannot delete yourself.\n");
        return;
    }
    if (u->role == ROLE_LIBRARIAN) {
        printf("A librarian account cannot be deleted.\n");
        return;
    }
    if (userHasActiveLoans(username)) {
        printf("This user still has books. They must return them first.\n");
        return;
    }
    deleteUserNode(username);
    printf("User deleted.\n");
}

/* Ask for username + password, then open the right menu */
void login(void)
{
    char username[30], password[30];
    readLine("Username: ", username, sizeof(username));
    readLine("Password: ", password, sizeof(password));

    User *u = findUser(username);
    if (u == NULL || strcmp(u->password, password) != 0) {
        printf("Wrong username or password.\n");
        return;
    }
    currentUser = u;
    printf("\nWelcome, %s! You are logged in as %s.\n", u->name, roleName(u->role));

    switch (u->role) {
        case ROLE_STUDENT: studentMenu();   break;
        case ROLE_STAFF:   staffMenu();     break;
        default:           librarianMenu(); break;
    }

    currentUser = NULL;
    printf("Logged out.\n");
}
```

A.7 cart.c

File: cart.c (lines 1-107)

```
/*
 * cart.c - the CART LINKED LIST (only for the logged-in student)
 */
#include "library.h"

CartItem *cartHead = NULL;   /* first node of the cart */

int cartContains(int bookId)
{
    CartItem *temp = cartHead;
    while (temp != NULL) {
        if (temp->bookId == bookId) return 1;
        temp = temp->next;
    }
    return 0;
}

/* Insert at the BEGINNING of the cart list (fastest insert) */
void addToCart(void)
{
    int id = readInt("Enter book ID to add to cart: ");
    Book *b = findBook(id);
    if (b == NULL) {
        printf("Book not found.\n");
        return;
    }
    if (cartContains(id)) {
        printf("This book is already in your cart.\n");
        return;
    }
    if (b->available == 0) {
        printf("Note: no copy is free right now, but you can still request it.\n");
    }
    CartItem *node = newNode(sizeof(CartItem));
    node->bookId = id;
    node->next = cartHead;   /* new node points to old first node */
    cartHead = node;         /* new node becomes the first node   */
    printf("\"%s\" added to cart.\n", b->title);
}

void removeFromCart(void)
{
    int id = readInt("Enter book ID to remove from cart: ");
    CartItem *temp = cartHead, *prev = NULL;
    while (temp != NULL && temp->bookId != id) {
        prev = temp;
        temp = temp->next;
    }
    if (temp == NULL) {
        printf("That book is not in your cart.\n");
        return;
    }
    if (prev == NULL) cartHead = temp->next;
    else prev->next = temp->next;
    free(temp);
    printf("Removed from cart.\n");
}

void showCart(void)
{
    if (cartHead == NULL) {
        printf("\nYour cart is empty.\n");
        return;
    }
    printBookHeader();
    CartItem *temp = cartHead;
    while (temp != NULL) {
        Book *b = findBook(temp->bookId);
        if (b != NULL) printBook(b);
        temp = temp->next;
    }
}

/* Free every node of the cart (used after submit and on logout) */
```

```
 75  void clearCart(void)
 76  {
 77      while (cartHead != NULL) {
 78          CartItem *temp = cartHead;
 79          cartHead = cartHead->next;
 80          free(temp);
 81      }
 82  }
 83
 84  /* Every book in the cart becomes a request in the queue */
 85  void submitCart(void)
 86  {
 87      if (cartHead == NULL) {
 88          printf("Your cart is empty. Add books first.\n");
 89          return;
 90      }
 91      CartItem *temp = cartHead;
 92      while (temp != NULL) {
 93          Book *b = findBook(temp->bookId);
 94          if (b == NULL) {
 95              printf("Book %d no longer exists - skipped.\n", temp->bookId);
 96          } else if (hasPendingRequest(currentUser->username, temp->bookId)) {
 97              printf("You already requested \"%s\" - skipped.\n", b->title);
 98          } else if (hasActiveLoan(currentUser->username, temp->bookId)) {
 99              printf("You already have \"%s\" - skipped.\n", b->title);
100          } else {
101              enqueue(nextRequestId, currentUser->username, temp->bookId);
102              printf("Request sent for \"%s\".\n", b->title);
103          }
104          temp = temp->next;
105      }
106      clearCart();
107  }
```

A.8 request.c

File: request.c (lines 1-108)

```
  1  /*
  2   * request.c - the BORROW REQUEST QUEUE
  3   *             (a linked list: enqueue at rear, dequeue at front)
  4   */
  5  #include "library.h"
  6
  7  Request *queueFront    = NULL;   /* remove from here */
  8  Request *queueRear     = NULL;   /* add here         */
  9  int      nextRequestId = 1;      /* id for the next new request */
 10
 11  void enqueue(int id, const char *username, int bookId)
 12  {
 13      Request *node = newNode(sizeof(Request));
 14      node->id = id;
 15      strcpy(node->username, username);
 16      node->bookId = bookId;
 17      node->next = NULL;
 18
 19      if (queueRear == NULL) {         /* queue is empty */
 20          queueFront = node;
 21          queueRear = node;
 22      } else {
 23          queueRear->next = node;      /* join at the back of the line */
 24          queueRear = node;
 25      }
 26      if (id >= nextRequestId) nextRequestId = id + 1;
 27  }
 28
 29  /* Remove and return the request at the front. Caller must free() it. */
 30  Request *dequeue(void)
 31  {
 32      if (queueFront == NULL) return NULL;
 33      Request *node = queueFront;
 34      queueFront = queueFront->next;
 35      if (queueFront == NULL) queueRear = NULL;   /* queue became empty */
 36      return node;
 37  }
 38
 39  /* Show pending requests. username == NULL means "show everyone's". */
 40  void showRequests(const char *username)
 41  {
 42      int count = 0;
 43      Request *temp = queueFront;
 44      while (temp != NULL) {
 45          if (username == NULL || strcmp(temp->username, username) == 0) {
 46              if (count == 0) {
 47                  printf("\n%-6s %-15s %-8s %-32s\n", "ReqID", "Student", "BookID", "Title");
 48                  printf("-----------------------------------------------------------------\n");
 49              }
 50              Book *b = findBook(temp->bookId);
 51              printf("%-6d %-15s %-8d %-32.32s\n", temp->id, temp->username,
 52                     temp->bookId, b ? b->title : "(deleted book)");
 53              count++;
 54          }
 55          temp = temp->next;
 56      }
 57      if (count == 0) printf("\nNo pending requests.\n");
 58  }
 59
 60  int hasPendingRequest(const char *username, int bookId)
 61  {
 62      Request *temp = queueFront;
 63      while (temp != NULL) {
 64          if (strcmp(temp->username, username) == 0 && temp->bookId == bookId) return 1;
 65          temp = temp->next;
 66      }
 67      return 0;
 68  }
 69
 70  /* Staff: take requests from the FRONT of the queue one by one */
 71  void processRequests(void)
 72  {
 73      while (queueFront != NULL) {
 74          Request *r = queueFront;
```

```
        Book *b = findBook(r->bookId);

        printf("\nNext request  #%d\n", r->id);
        printf("  Student : %s\n", r->username);
        printf("  Book    : %d - %s\n", r->bookId, b ? b->title : "(deleted book)");
        printf("  Copies available: %d\n", b ? b->available : 0);
        printf("1. Approve   2. Reject   0. Stop\n");

        switch (readInt("Choice: ")) {
            case 1:                                 /* approve */
                if (b == NULL || b->available == 0) {
                    printf("Cannot approve: no copy available. Reject it or try later.\n");
                    break;
                }
                r = dequeue();
                b->available--;                     /* reserve one copy */
                addLoan(r->id, r->username, r->bookId, STATUS_APPROVED);
                printf("Approved. Student can now check out the book.\n");
                free(r);
                break;
            case 2:                                 /* reject */
                r = dequeue();
                printf("Request rejected.\n");
                free(r);
                break;
            case 0:                                 /* stop */
                return;
            default:
                printf("Invalid choice.\n");
        }
        saveAll();
    }
    printf("\nNo more pending requests.\n");
}
```

A.9 loan.c

File: loan.c (lines 1-125)

```
/*
 * loan.c - the LOAN LINKED LIST, check-out and check-in
 */
#include "library.h"

Loan *loanHead = NULL;   /* first node of the loan list */

void addLoan(int id, const char *username, int bookId, int status)
{
    Loan *node = newNode(sizeof(Loan));
    node->id = id;
    strcpy(node->username, username);
    node->bookId = bookId;
    node->status = status;
    node->next = NULL;

    if (loanHead == NULL) {
        loanHead = node;
    } else {
        Loan *temp = loanHead;
        while (temp->next != NULL) temp = temp->next;
        temp->next = node;
    }
    if (id >= nextRequestId) nextRequestId = id + 1;
}

Loan *findLoan(int id)
{
    Loan *temp = loanHead;
    while (temp != NULL) {
        if (temp->id == id) return temp;
        temp = temp->next;
    }
    return NULL;
}

/* Returns 1 if this user still has this book (approved or borrowed) */
int hasActiveLoan(const char *username, int bookId)
{
    Loan *temp = loanHead;
    while (temp != NULL) {
        if (strcmp(temp->username, username) == 0 && temp->bookId == bookId &&
            temp->status != STATUS_RETURNED) return 1;
        temp = temp->next;
    }
    return 0;
}

/* Returns 1 if this user still has ANY book (approved or borrowed) */
int userHasActiveLoans(const char *username)
{
    Loan *temp = loanHead;
    while (temp != NULL) {
        if (strcmp(temp->username, username) == 0 && temp->status != STATUS_RETURNED) return 1;
        temp = temp->next;
    }
    return 0;
}

/* Show loans. username == NULL means everyone. onlyActive == 1 hides returned ones. */
void showLoans(const char *username, int onlyActive)
{
    int count = 0;
    Loan *temp = loanHead;
    while (temp != NULL) {
        int userOk   = (username == NULL || strcmp(temp->username, username) == 0);
        int statusOk = (!onlyActive || temp->status != STATUS_RETURNED);
        if (userOk && statusOk) {
            if (count == 0) {
                printf("\n%-7s %-15s %-8s %-28s %-18s\n", "LoanID", "Student", "BookID", "Title", "Status");
                printf("-------------------------------------------------------------------------------\n");
            }
            Book *b = findBook(temp->bookId);
            printf("%-7d %-15s %-8d %-28.28s %-18s\n", temp->id, temp->username, temp->bookId,
```

```
                   b ? b->title : "(deleted book)", statusName(temp->status));
            count++;
        }
        temp = temp->next;
    }
    if (count == 0) printf("\nNo loans found.\n");
}

/* Student picks up an approved book */
void checkOutBook(void)
{
    showLoans(currentUser->username, 1);
    int id = readInt("Enter Loan ID to check out (pick up): ");
    Loan *loan = findLoan(id);
    if (loan == NULL || strcmp(loan->username, currentUser->username) != 0) {
        printf("No such loan for you.\n");
        return;
    }
    if (loan->status != STATUS_APPROVED) {
        printf("This loan is not waiting for pick up.\n");
        return;
    }
    char date[20];
    getDate(date, sizeof(date));
    loan->status = STATUS_BORROWED;
    pushHistory(currentUser->username, loan->bookId, "CHECKOUT", date);
    printf("Book checked out. Enjoy reading!\n");
}

/* Student returns a borrowed book */
void checkInBook(void)
{
    showLoans(currentUser->username, 1);
    int id = readInt("Enter Loan ID to check in (return): ");
    Loan *loan = findLoan(id);
    if (loan == NULL || strcmp(loan->username, currentUser->username) != 0) {
        printf("No such loan for you.\n");
        return;
    }
    if (loan->status != STATUS_BORROWED) {
        printf("This book is not checked out.\n");
        return;
    }
    char date[20];
    getDate(date, sizeof(date));
    loan->status = STATUS_RETURNED;
    Book *b = findBook(loan->bookId);
    if (b != NULL) b->available++;          /* copy goes back on the shelf */
    pushHistory(currentUser->username, loan->bookId, "CHECKIN", date);
    printf("Book checked in. Thank you!\n");
}
```

A.10 history.c

File: history.c (lines 1-51)

```
 1  /*
 2   * history.c - the HISTORY STACK (a linked list: push on top, newest first)
 3   */
 4  #include "library.h"
 5
 6  History *historyTop = NULL;   /* top of the history stack */
 7
 8  void pushHistory(const char *username, int bookId, const char *action, const char *date)
 9  {
10      History *node = newNode(sizeof(History));
11      strcpy(node->username, username);
12      node->bookId = bookId;
13      strcpy(node->action, action);
14      strcpy(node->date, date);
15      node->next = historyTop;   /* new node sits on top of old top */
16      historyTop = node;
17  }
18
19  /* Show history from top of stack (newest) to bottom (oldest).
20   * username == NULL means "show everyone's". */
21  void showHistory(const char *username)
22  {
23      int count = 0;
24      History *temp = historyTop;
25      while (temp != NULL) {
26          if (username == NULL || strcmp(temp->username, username) == 0) {
27              if (count == 0) {
28                  printf("\n%-17s %-15s %-9s %-8s %-25s\n", "Date", "Student", "Action", "BookID", "Title");
29                  printf("-------------------------------------------------------------------------------------\n");
30              }
31              Book *b = findBook(temp->bookId);
32              printf("%-17s %-15s %-9s %-8d %-25.25s\n", temp->date, temp->username,
33                     temp->action, temp->bookId, b ? b->title : "(deleted book)");
34              count++;
35          }
36          temp = temp->next;
37      }
38      if (count == 0) printf("\nNo history found.\n");
39  }
40
41  /* Staff: see the history of one student, or of everyone */
42  void viewStudentHistory(void)
43  {
44      char username[30];
45      readLine("Enter student username (or ALL): ", username, sizeof(username));
46      if (strcmp(username, "ALL") == 0 || strcmp(username, "all") == 0) {
47          showHistory(NULL);
48      } else {
49          showHistory(username);
50      }
51  }
```

A.11 file.c

File: file.c (lines 1-130)

```
/*
 * file.c - FILE HANDLING (save and load everything)
 *          Each line of a file is one node. Fields are separated by '|'
 */
#include "library.h"

void saveAll(void)
{
    FILE *f;

    f = fopen(BOOK_FILE, "w");
    if (f != NULL) {
        for (Book *b = bookHead; b != NULL; b = b->next)
            fprintf(f, "%d|%s|%s|%d|%d\n", b->id, b->title, b->author, b->total, b->available);
        fclose(f);
    }

    f = fopen(USER_FILE, "w");
    if (f != NULL) {
        for (User *u = userHead; u != NULL; u = u->next)
            fprintf(f, "%s|%s|%s|%d\n", u->username, u->password, u->name, u->role);
        fclose(f);
    }

    f = fopen(REQUEST_FILE, "w");
    if (f != NULL) {
        for (Request *r = queueFront; r != NULL; r = r->next)
            fprintf(f, "%d|%s|%d\n", r->id, r->username, r->bookId);
        fclose(f);
    }

    f = fopen(LOAN_FILE, "w");
    if (f != NULL) {
        for (Loan *l = loanHead; l != NULL; l = l->next)
            fprintf(f, "%d|%s|%d|%d\n", l->id, l->username, l->bookId, l->status);
        fclose(f);
    }

    /* History is saved from top (newest) to bottom (oldest) */
    f = fopen(HISTORY_FILE, "w");
    if (f != NULL) {
        for (History *h = historyTop; h != NULL; h = h->next)
            fprintf(f, "%s|%d|%s|%s\n", h->username, h->bookId, h->action, h->date);
        fclose(f);
    }
}

/* Returns 1 if this is the very first run (no users file yet) */
int loadAll(void)
{
    FILE *f;
    char line[300];

    f = fopen(USER_FILE, "r");
    if (f == NULL) return 1;              /* first run */
    while (fgets(line, sizeof(line), f)) {
        char username[30], password[30], name[60];
        int role;
        if (sscanf(line, "%29[^|]|%29[^|]|%59[^|]|%d", username, password, name, &role) == 4)
            addUser(username, password, name, role);
    }
    fclose(f);

    f = fopen(BOOK_FILE, "r");
    if (f != NULL) {
        while (fgets(line, sizeof(line), f)) {
            int id, total, available;
            char title[100], author[60];
            if (sscanf(line, "%d|%99[^|]|%59[^|]|%d|%d", &id, title, author, &total, &available) == 5)
                addBook(id, title, author, total, available);
        }
        fclose(f);
    }

```

```
    f = fopen(REQUEST_FILE, "r");
    if (f != NULL) {
        while (fgets(line, sizeof(line), f)) {
            int id, bookId;
            char username[30];
            if (sscanf(line, "%d|%29[^|]|%d", &id, username, &bookId) == 3)
                enqueue(id, username, bookId);
        }
        fclose(f);
    }

    f = fopen(LOAN_FILE, "r");
    if (f != NULL) {
        while (fgets(line, sizeof(line), f)) {
            int id, bookId, status;
            char username[30];
            if (sscanf(line, "%d|%29[^|]|%d|%d", &id, username, &bookId, &status) == 4)
                addLoan(id, username, bookId, status);
        }
        fclose(f);
    }

    /* The file is newest-first. If we pushed each line, the oldest would end
     * up on top (wrong order). So we add each line at the BOTTOM instead. */
    f = fopen(HISTORY_FILE, "r");
    if (f != NULL) {
        History *bottom = NULL;
        while (fgets(line, sizeof(line), f)) {
            History *node = newNode(sizeof(History));
            if (sscanf(line, "%29[^|]|%d|%9[^|]|%19[^\n]", node->username, &node->bookId,
                       node->action, node->date) != 4) {
                free(node);
                continue;
            }
            node->next = NULL;
            if (bottom == NULL) historyTop = node;
            else bottom->next = node;
            bottom = node;
        }
        fclose(f);
    }
    return 0;
}

/* First run: create the main librarian and a few sample books */
void createDefaultData(void)
{
    addUser("admin", "admin123", "Head Librarian", ROLE_LIBRARIAN);
    addBook(nextBookId, "C Programming Language", "Kernighan & Ritchie", 3, 3);
    addBook(nextBookId, "Data Structures in C", "Reema Thareja", 2, 2);
    addBook(nextBookId, "Introduction to Algorithms", "Cormen", 2, 2);
    addBook(nextBookId, "Let Us C", "Yashavant Kanetkar", 4, 4);
    addBook(nextBookId, "Clean Code", "Robert C. Martin", 1, 1);
    saveAll();
    printf("First run: default librarian created (admin / admin123).\n");
}
```